BuildScan 3D: Mobile-to-Cloud 3D Gaussian Splatting for Metric Indoor Reconstruction

Project Team

Muhammad Umair 23I-0662
Zaid Fareed 23I-0829
Muhammad Tayyab 23I-0636

Session 2023-2027

Supervised by

Mr. Furqan Nasir



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Contents

Students' Submission i

Anti-Plagiarism Statement ii

Authors' Declaration iii

Abstract iv

Acknowledgments v

Executive Summary vi

List of Abbreviations viii

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 2
1.5 Project Objectives . . . 2
1.6 Project Scope . . . 3
1.7 Project Modules . . . 4
1.8 Stakeholders and User Classes . . . 4
1.9 Success Criteria . . . 4
1.10 Constraints and Assumptions . . . 5
1.11 Ethical, Legal and DEI Considerations . . . 5

2 Research and Existing Solutions 7

2.1 Research Method / Approach . . . 7
2.2 Existing Solutions / Products . . . 7
2.3 Comparison of Existing Solutions . . . 8
2.4 Limitations and Gaps . . . 8
2.5 Proposed Solution . . . 9
2.6 Complex Computing Problem . . . 10

2.7 Technical Challenges . . . 10
2.8 Research Findings and Design Decisions . . . 11
2.9 Techniques Explored and Parked . . . 12

3 Proposal Feedback and Iteration Plan 14
3.1 Proposal Feedback Summary . . . 14
3.2 Proposal Action Register . . . 14
3.3 Iteration Strategy . . . 15
3.4 Iteration 1 Plan . . . 15
3.5 Iteration 2 Plan . . . 16
3.6 Work Breakdown and Responsibilities . . . 16
3.7 Progress Against Plan . . . 16
3.8 Delays, Deviations and Recovery Plan . . . 17

4 Software Requirements 19
4.1 User Interactions / Use Cases . . . 19
4.2 Use Case Diagram . . . 19
4.3 Detailed Use Cases . . . 20
4.4 Functional Requirements . . . 21
4.4.1 Mobile Capture Module . . . 21
4.4.2 Processing Server Module . . . 21
4.4.3 Reconstruction Pipeline Module . . . 22
4.5 Non-Functional Requirements . . . 22
4.5.1 Reliability . . . 22
4.5.2 Usability . . . 22
4.5.3 Performance . . . 23
4.5.4 Security . . . 23
4.6 Acceptance Criteria . . . 23
4.7 Requirements Traceability . . . 23

5 System Design 25
5.1 System Architecture . . . 25
5.2 Architectural Components . . . 25
5.3 Interfaces and Dependencies . . . 27
5.4 Architecture Decisions and Rationale . . . 28
5.5 Design Evolution . . . 28
5.6 Domain Model . . . 29
5.7 Data Design . . . 29
5.8 Process / Behavior Design . . . 30

6 Iteration 1: Design and Implementation 34

6.1 Iteration Objectives 34
6.2 Selected Use Cases / Features 34
6.3 Design 34
6.4 Implementation 35
6.5 Completed Features / Module 35
6.6 Technical Implementation Details 37
6.7 Algorithms and Technical Decisions 38
6.8 Tools, Frameworks and Technologies 39
6.9 Demonstration Evidence 39
6.10 Contributions and Novelty 39
6.11 Iteration Outcome 40

7 Testing and Early Validation **42**
7.1 Testing Strategy 42
7.2 Test Levels and Techniques 43
7.3 Test Cases 43
7.4 Test Results 45
7.5 Acceptance Testing / Validation 46
7.6 Edge and Failure Cases 47
7.7 Defects and Corrective Actions 48
7.8 Current Validation Status 49

8 Project Management and Development Evidence **51**
8.1 Work Completed 51
8.2 Work Remaining 51
8.3 Individual Responsibilities 52
8.4 Contribution of Each Team Member 52
8.5 Repository and Version Control Evidence 53
8.6 Milestones and Commit Evidence 53
8.7 Tools and Tool-Assisted Development 53
8.8 External Components and Dependencies 54
8.9 Risks and Mitigation 54
8.10 Security and Privacy Considerations 56
8.11 Current Limitations 57

9 Remaining FYP Plan **61**
9.1 Remaining Scope 61
9.2 Remaining Modules and Features 61
9.3 Remaining Iterations 62
9.4 Implementation Plan 63
9.5 Testing and Validation Plan 64

9.6 Deployment Plan . . . 64
9.7 Tentative Timeline . . . 65
9.8 Expected Final Deliverables . . . 65

10 Conclusion **66**

A Proposal Action Register **68**

B Detailed Requirements **71**
B.1 Extended Functional Requirements . . . 71
B.1.1 Mobile Capture Module — Detailed . . . 71
B.1.2 Processing Server Module — Detailed . . . 72
B.1.3 Reconstruction Pipeline Module — Detailed . . . 72
B.2 Extended Non-Functional Requirements . . . 73
B.3 Full Traceability Matrix . . . 73

C Detailed Test Cases **75**
C.1 Automated test files . . . 75
C.2 Test cases . . . 77

D Repository and Contribution Evidence **81**
D.1 Repository Structure . . . 81
D.2 Commit History Summary . . . 82
D.3 Branch Strategy . . . 82
D.4 CI/CD Evidence . . . 83
D.5 Task Tracking . . . 83

E Additional Design Diagrams **84**
E.1 Pipeline Step Sequence . . . 84
E.2 Android App Architecture . . . 84
E.3 Sim3 Alignment . . . 84
E.4 Coverage Map . . . 84

F Meeting and Progress Evidence **88**
F.1 Supervisor Meetings . . . 88
F.2 Progress Log . . . 88
F.3 Problems and Solutions Summary . . . 89

List of Figures

4.1 Use case diagram. The End User is the primary actor; the Processing Server (`server/main.py`) and the Cloud GPU (RunPod pod, or the local GPU) are secondary actors. Take Measurement currently runs in the desktop viewer; the web version is FYP-2 work. 20

5.1 System architecture. Client: Android app (`googleARCore/samples/hello_ar_java/`) and the desktop viewer. Server: `server/main.py`, `pipeline.py`, `runpod_client.py`. Compute: the local GPU, or a RunPod pod reached through the RunPod GraphQL API and SSH/SCP. Both compute paths run the same six steps; `test_server_pipeline.py` checks that the two command lists match. . 26

5.2 Job state machine (`JobStatus` in `server/pipeline.py`). A timeout is enforced per step by the watchdog loop in `run_pipeline()`, which kills the subprocess and marks the job failed; cancel is `POST /jobs/{job_id}/cancel`. In RunPod mode a separate pod watchdog terminates the pod after `MAX_POD_LIFETIME` (default 7200 s). An `uploading` value is defined in `JobStatus` but no code path sets it. 29

5.3 Data flow from the capture files to the model, as run by `server/pipeline.py` (and, step for step, on a RunPod pod). Steps 1–2 read the capture through `common.Capture`; `fallback.py` reads the IMU samples from the MP4's own data tracks (`imu.py`). 31

5.4 Sequence diagram for upload → process → download in RunPod mode (`ProcessingActivity.java`, `server/main.py`, `run_pipeline_on_runpod()` in `server/runpod_client.py`). The app polls every 3 s (5 s after an error) and stops when the status is `completed` or `failed`. The app does not download the model; it is fetched with `GET /download` by the desktop viewer. 33

E.1 The six pipeline steps and their timeouts (`STEPS` in `server/pipeline.py`). The measured end-to-end time on the development RTX 3070 is ∼75 min; no cloud-GPU time has been measured yet. 85

E.2 Android capture app (`googleARCore/samples/hello_ar_java/.../helloar/`). Arrows from `HelloArActivity` show ownership or navigation (`Intents`); `CaptureLogger` feeds raw depth to `CoverageMap`. `ProcessingActivity` uploads six files plus a zipped `depth/` folder; `camera.csv` and `anchors.csv` stay on the phone. 86

E.3 Sim3 alignment of the COLMAP reconstruction to ARCore's metric frame (`sfm_colmap.py`, export stage; Umeyama fit unit-tested in `test_relocalize_and_sfm.py`). Because ARCore re-anchors silently, only its largest self-consistent segment ends up as inliers. Inlier median residual: about 2.5 cm. The resulting scale is validated against a tape measure (Table 7.5). 86

E.4 Live coverage map (`CoverageMap.java`, `CoverageOverlayView.java`). ARCore raw depth (5 Hz, confidence $\geq$128, up to 5 m) is accumulated in 10 cm voxels; each voxel records from which of 16 directions it was seen. Visible voxels are drawn as dots, coloured by direction count, with at most 5,000 voxels projected and 2,000 dots drawn, refreshed at 2 Hz. The grid colouring is illustrative. 87

List of Tables

2.1 Comparison of existing 3D scanning solutions 8
2.2 Techniques explored and parked 12
2.2 Techniques explored and parked 13

3.2 Work breakdown and ownership 17

4.1 Acceptance criteria for key requirements 24
4.2 Requirements traceability matrix (selected) 24

5.1 Key interfaces between components 27

6.1 IMU fallback on simulated tracking gaps (position error, median / max) . 37
6.2 Tools and technologies used 39

7.1 Test levels and techniques 43
7.2 Test cases and their status 43
7.2 Test cases and their status 44
7.2 Test cases and their status 45
7.3 Reconstruction quality on the first room: 42 held-out frames that no model was trained on (`fair_compare.py`) 46
7.4 Before and after replacing ARCore poses with image-based (COLMAP) poses 46
7.5 Metric accuracy: model versus tape measure (rigid objects) 47

8.5 Risk register 54
8.5 Risk register 55
8.5 Risk register 56
8.1 Owned modules per team member (from the commit history of each file) . 58
8.2 Key milestones with commit evidence 59
8.3 AI and automated tools used during development 60
8.4 External components and their licenses 60

9.1 FYP-2 tentative timeline 65

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *Build-Scan 3D: Mobile-to-Cloud 3D Gaussian Splatting for Metric Indoor Reconstruction*.

Student 1: Muhammad Umair (23I-0662) Signature: _____

Student 2: Zaid Fareed (23I-0829) Signature: _____

Student 3: Muhammad Tayyab (23I-0636) Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable.

Abstract

Existing 3D scanning solutions for indoor environments require expensive LiDAR hardware or multi-step manual workflows inaccessible to construction site teams, facility managers, and small architectural firms. This project, **BuildScan 3D**, addresses the gap by providing an end-to-end pipeline that converts a recording from a commodity Android smartphone (a realme phone, with no LiDAR or special hardware) into a metric-scale, photorealistic 3D Gaussian Splatting (3DGS) model suitable for real-world measurements.

The system comprises five stages: (1) a custom ARCore-based Android application that captures 30 FPS video alongside camera poses, depth maps, and live image-quality feedback; (2) a pose-prior module that aligns ARCore's OpenGL coordinates to the COLMAP/Nerfstudio convention; (3) a cloud-GPU reconstruction pipeline that performs COLMAP Structure-from-Motion with Sim3 alignment to ARCore's metric frame, followed by Nerfstudio `splatfacto` 3D Gaussian Splatting training; (4) a structural layer that extracts wall planes, corners, wall lengths and doors from the model as the basis for floor plans and 1:1 measurement (prototype built); and (5) a planned web-based viewer for metric measurement and model inspection.

At mid-evaluation, Iteration 1 stands at *core pipeline complete, refinement ongoing*: the Android capture app records all required sensor data with live quality and coverage guidance; the FastAPI processing server accepts uploads and dispatches jobs to on-demand RunPod GPU pods; and the current best reconstruction (v6) scores 26.52 dB PSNR / 0.8886 SSIM on 42 held-out frames, and tape-measure checks show a mean measurement error of 4.8 mm (0.19%), worst case 0.29%, against a 5% accuracy target. Key technical challenges solved include IMU-bridged tracking-gap recovery, lens self-calibration via COLMAP, and diagnosis and correction of degenerate Gaussian splat geometry. The open items are 3DGS refinement, 3D mesh creation, completion of the structural layer, and the web workspace (including floor plans and 1:1 measurement), followed by end-to-end field validation.

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Mr. Furqan Nasir, for his continuous guidance, constructive feedback, and encouragement throughout this project. His insights on system architecture and research methodology have been invaluable.

We also thank the faculty of the FAST School of Computing, National University of Computer and Emerging Sciences, Islamabad, for providing the academic foundation and resources that made this work possible.

Finally, we acknowledge the open-source communities behind ARCore, COLMAP, Nerfstudio, and 3D Gaussian Splatting, whose publicly available tools and research form the backbone of our reconstruction pipeline.

Executive Summary

BuildScan 3D is a mobile-to-cloud system that converts a short smartphone video of an indoor space into a photorealistic, metric-scale 3D model suitable for measurements, facility documentation, and construction progress tracking.

Problem. Professional 3D scanning of indoor spaces requires expensive LiDAR equipment ($10,000+) or photogrammetry workflows that demand expert knowledge and hours of manual processing. Small construction firms, facility managers, and architectural students lack access to affordable, automated tools that produce models accurate enough for real-world measurements.

Research. We reviewed the 3D Gaussian Splatting literature (1), on-device reconstruction (PocketGS (2)), and visual-inertial SLAM approaches (VIGS-SLAM (3)). The key gap we identified is that no existing system combines smartphone ARCore capture with cloud-GPU 3DGS processing to deliver metric-scale models with sub-centimetre measurement accuracy in a fully automated pipeline.

Solution. Our five-stage pipeline: (1) an Android app captures video, ARCore poses, depth and IMU data at 30 FPS with live quality guidance; (2) pose conversion and coordinate alignment; (3) cloud GPU processing via COLMAP SfM and Nerfstudio splatfacto 3DGS; (4) a structural layer (wall planes, corners, wall lengths, door detection) for floor plans and 1:1 measurement, prototype built; (5) planned Three.js web viewer with metric measurements.

Current status (Iteration 1: core pipeline complete, refinement ongoing). The Android capture app records all required sensor data with live image-quality assessment and coverage guidance (red/yellow/green dot overlay). The FastAPI processing server accepts uploads and dispatches jobs to on-demand RunPod GPU pods (NVIDIA RTX 4090 by default, selectable at runtime), with automatic pod termination on completion or failure. End-to-end processing has been measured at about 75 minutes on the development RTX 3070; no cloud-GPU timing has been measured yet, and the under-35-minute RTX

4090 figure is an FYP-2 projection. The current best reconstruction (v6) scores 26.52 dB PSNR / 0.8886 SSIM on 42 held-out frames that no model was trained on. Tape-measure checks show a mean error of 4.8 mm (0.19%) and a worst case of 0.29% (8 mm over a 2.8 m wall), far inside the 5% accuracy target. CI/CD via GitHub Actions validates Python code, Android builds, and web UI on every push. Open items: 3DGS refinement, 3D mesh creation, completion of the structural layer, and the web workspace.

Risks. Cloud GPU availability and per-job cost (not yet measured); structural-layer robustness across rooms (ceiling-height detection is not yet reliable); measurement accuracy validation across diverse room geometries; the unmeasured RTX 4090 processing time. The full risk register has twelve entries with likelihood, impact, mitigation and owner.

FYP-2 plan. Complete the structural layer and automatic floor plans (Stage 4), build the web measurement workspace (Stage 5), conduct field trials on more rooms, and validate measurement accuracy against tape-measure ground truth across multiple rooms. 3D mesh creation is also in scope as planned work; it has not yet been scheduled.

Contents

List of Tables

List of Figures

List of Abbreviations

3DGS	3D Gaussian Splatting
API	Application Programming Interface
AR	Augmented Reality
ARCore	Google's AR Platform for Android
BA	Bundle Adjustment
CI/CD	Continuous Integration / Continuous Delivery
COLMAP	Structure-from-Motion and Multi-View Stereo Pipeline
dB	Decibels
FPS	Frames Per Second
FR	Functional Requirement
FYP	Final Year Project
GPU	Graphics Processing Unit
IMU	Inertial Measurement Unit
IQA	Image Quality Assessment
LiDAR	Light Detection and Ranging
MVS	Multi-View Stereo
NFR	Non-Functional Requirement
PLY	Polygon File Format
PSNR	Peak Signal-to-Noise Ratio
SCP	Secure Copy Protocol
SfM	Structure from Motion
SIFT	Scale-Invariant Feature Transform
Sim3	Similarity Transform (rotation + translation + scale)
SO(3)	Special Orthogonal Group in 3D
SSH	Secure Shell
SSIM	Structural Similarity Index Measure
TSDF	Truncated Signed Distance Function
VRAM	Video Random Access Memory

Chapter 1

Introduction and Project Vision

1.1 Project Overview

BuildScan 3D is an end-to-end system that enables non-expert users to create photorealistic, metric-scale 3D models of indoor spaces using only a commodity Android smartphone. The user records a short walk-through video; the system automatically uploads the captured data to a cloud GPU, reconstructs a 3D Gaussian Splatting model, and returns a viewable, measurable digital twin of the scanned space.

The system targets construction site documentation, facility management, and architectural surveying—domains where 3D capture is valuable but traditional solutions (terrestrial LiDAR scanners, professional photogrammetry) are prohibitively expensive or require specialist operators.

1.2 Problem Statement

Capturing accurate, metric-scale 3D models of indoor environments currently requires either (a) LiDAR hardware costing $10,000 or more, (b) photogrammetry workflows requiring expert knowledge and hours of manual post-processing, or (c) consumer-grade scanning apps (e.g. Polycam, Luma AI) that produce visually plausible but metrically inaccurate models unsuitable for measurement. There is no affordable, fully automated pipeline that turns a smartphone video into a metrically accurate 3D model with sufficient quality for professional use.

1.3 Problem Context and Motivation

Construction and facility management lose time and money to rework caused by inaccurate or outdated spatial documentation. Small firms and site teams need quick, accurate as-built 3D captures but cannot justify the cost of LiDAR equipment or trained surveyors for routine inspections. Modern smartphones carry high-resolution cameras, IMUs, and AR-capable processors (via Google ARCore), making them potential 3D capture devices—if the software pipeline can bridge the accuracy gap between consumer AR and professional surveying.

Recent advances in 3D Gaussian Splatting (1) enable photorealistic novel-view synthesis at real-time frame rates from posed images, while ARCore provides metric-scale camera poses and depth sensing on Android devices. Combining these technologies with cloud GPU processing creates an opportunity for a fully automated, affordable indoor scanning solution.

1.4 Project Vision

BuildScan 3D aims to democratize indoor 3D scanning by making it as simple as recording a video on a phone. The system will: (1) guide the user during capture to ensure sufficient coverage and quality; (2) automatically process the recording on a cloud GPU; (3) return a metric-scale 3D model with measurement tools; and (4) extract the room's structure—wall planes, corners, wall lengths and doors—as the basis for automatic floor plans and 1:1 metric measurement.

1.5 Project Objectives

1. Develop an Android capture application that records video, ARCore poses, depth maps, and IMU data with live image-quality feedback.

2. Build an automated cloud-GPU processing pipeline that converts a phone recording into a 3D Gaussian Splatting model, targeting under 35 minutes end-to-end on a cloud RTX 4090 (a projection; the measured time today is $\sim$75 minutes on the development RTX 3070).

3. Achieve metric-scale accuracy within 5% of tape-measure ground truth for distances measured in the model.

4. Achieve held-out PSNR $\geq$ 25 dB for photometric reconstruction quality.

5. Build a structural layer that extracts wall planes, corners, wall lengths and doors from the reconstructed model, as the basis for automatic floor plans and 1:1 metric measurement.

6. Build a web-based 3D viewer with point-to-point metric measurement tools.

1.6 Project Scope

In scope:

- Android capture application with ARCore integration, live IQA, and coverage guidance.
- Automated reconstruction pipeline: keyframe extraction, COLMAP SfM, Sim3 metric alignment, splatfacto 3DGS training.
- FastAPI processing server with local and cloud GPU (RunPod) execution modes.
- Metric measurement capability in the 3D model.
- Structural layer: automatic wall planes, corners, wall lengths, door detection and floor plans (prototype built in FYP-1; completion in FYP-2).
- Three.js/WebGPU web viewer for model inspection and measurement (FYP-2).
- 3D mesh creation from the reconstructed model (planned; not yet scheduled).
- CI/CD pipeline via GitHub Actions.

Out of scope:

- Outdoor environments and large-scale (building-wide) scans.
- iOS support (ARKit).
- Real-time on-device reconstruction.
- BIM (Building Information Modelling) export.

1.7 Project Modules

1. **Mobile Capture Module**: Android app using ARCore for video recording, camera pose tracking, depth sensing, IMU telemetry, and live quality/coverage guidance.
2. **Pose Prior Module**: Coordinate alignment from ARCore's OpenGL convention to COLMAP/Nerfstudio, quaternion-to-rotation-matrix conversion, and Sim3 metric alignment.
3. **Reconstruction Pipeline Module**: COLMAP SfM (feature extraction, matching, mapping, undistortion), lens self-calibration, and Nerfstudio splatfacto 3DGS training.
4. **Processing Server Module**: FastAPI backend for dataset upload, job management, progress tracking, and cloud GPU dispatch via RunPod API.
5. **Structural Layer Module**: Rebuilds the room shell next to the splat model (`structure.py`): floor and wall planes, corners, wall lengths, doors and other openings, and the floor polygon used for floor plans and 1:1 measurement. Prototype built; completion in FYP-2.
6. **Web Viewer Module** (FYP-2): Three.js/WebGPU-based viewer with splat rendering, point-to-point 1:1 measurement that can snap to structural-layer walls, and a floor-plan view.

1.8 Stakeholders and User Classes

- **Primary users**: Construction site managers, facility maintenance teams, architectural survey teams who need quick as-built documentation.
- **Secondary users**: Architecture and civil engineering students for educational purposes; real estate professionals for property documentation.
- **Administrators**: System operators who manage cloud GPU resources and monitor processing jobs.
- **Supervisor**: Mr. Furqan Nasir (academic evaluation and guidance).

1.9 Success Criteria

1. Metric accuracy: distances measured in the model within 5% of tape-measure ground truth (measured so far: mean 0.19%, worst case 0.29%).

2. Reconstruction quality: held-out PSNR ≥ 25 dB, SSIM ≥ 0.85.

3. Processing time: under 35 minutes end-to-end from upload to viewable model on an NVIDIA RTX 4090 cloud GPU (FYP-2 target, projected and not yet measured; currently $\sim$75 minutes on the development RTX 3070).

4. Capture guidance: live IQA reduces user-caused quality failures by $\geq$50% compared to unguided capture.

5. Structural layer: wall planes, wall lengths and doors extracted automatically, with wall lengths within the 5% accuracy target (FYP-2; not yet verified against a tape measure).

1.10 Constraints and Assumptions

- The target phone must support ARCore (Google's AR compatibility list). All results in this report come from a commodity Android phone (realme) with no LiDAR or other special hardware.
- Cloud GPU processing requires internet connectivity and a RunPod account.
- The scanned space must be a single room or connected set of rooms (not a full building).
- Indoor lighting must be sufficient for ARCore to maintain tracking (no pitch-dark environments).
- The current pipeline targets rooms up to approximately 10×10 m.

1.11 Ethical, Legal and DEI Considerations

- **Privacy**: The captured video may include personally identifiable spaces. All data is processed on the user's own cloud account; no data is shared with third parties. The system does not store or transmit video beyond what is needed for processing.
- **Accessibility**: The capture app provides visual quality feedback (colour-coded overlay) and haptic feedback (vibration on quality degradation) to assist users with varying levels of technical expertise.
- **Environmental**: Cloud GPU pods are terminated immediately after processing to minimize unnecessary compute resource usage and associated energy consumption.

- **Tool disclosure**: GenAI tools (Claude Code, GitHub Copilot, ChatGPT) were used for code generation assistance, debugging, documentation and report drafting, including the correction pass on this report. Full disclosure is provided in Chapter 8, Table 8.3.

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

Our research followed a structured literature review combined with empirical experimentation. We surveyed recent publications (2023–2025) in 3D Gaussian Splatting, on-device 3D reconstruction, and visual-inertial SLAM, focusing on methods that operate on monocular or smartphone-captured video. We then implemented and benchmarked candidate approaches on our own ARCore-captured datasets to validate claims and identify practical limitations. Key papers are archived in `docs/Research Papers/`.

2.2 Existing Solutions / Products

Commercial products:

- **Matterport Pro3**: Professional LiDAR scanner ($6,000+). Produces accurate metric models but requires dedicated hardware and per-scan cloud processing fees.
- **Polycam**: Smartphone app using LiDAR (iPhone Pro) or photogrammetry. Fast capture but limited metric accuracy on non-LiDAR devices; no automated cloud processing pipeline.
- **Luma AI**: Consumer 3DGS from video. Visually impressive but provides no metric scale and no measurement tools.
- **RealityCapture**: Professional photogrammetry software. High-quality output but requires expert operation, manual photo capture, and desktop GPU processing.

Research systems:

- **3D Gaussian Splatting** (1): Kerbl et al. (2023) introduced real-time radiance field rendering using anisotropic 3D Gaussians, achieving state-of-the-art novel-view synthesis quality. Requires pre-computed SfM poses; no built-in capture guidance or metric scale.

- **PocketGS** (2): On-device 3DGS training on a smartphone. Demonstrates that dense multi-view stereo initialization produces better geometry than sparse SfM points alone. Limited to short sequences and small scenes.

- **VIGS-SLAM** (3): Visual-inertial Gaussian splatting SLAM. Real-time tracking and mapping but requires custom hardware integration and is not available as a turnkey system.

- **PGSR** (4): Planar-based Gaussian Splatting for surface reconstruction. Improves geometric flatness of walls/floors but sacrifices reflective surface quality (glossy floors, mirrors).

2.3 Comparison of Existing Solutions

Table 2.1: Comparison of existing 3D scanning solutions

Criterion	**Matterport**	**Polycam**	**Luma AI**	**PocketGS**	**Ours**
Hardware cost	$6000+	Phone	Phone	Phone	Phone
Metric scale	Yes	Partial	No	No	Yes
Measurements	Yes	Basic	No	No	Yes
Auto processing	Yes	Yes	Yes	On-device	Cloud
Quality (PSNR)	N/A	Medium	High	High	High
Expert needed	No	No	No	No	No
Capture guidance	No	No	No	No	Yes
Floor plan from scan	Yes	LiDAR only	No	No	Prototype

2.4 Limitations and Gaps

The key limitations of existing solutions that motivate our work are:

1. **Cost barrier**: Professional solutions (Matterport, RealityCapture) require expensive hardware or software licenses.

2. **No metric scale from video**: Consumer apps (Luma AI) and research methods (3DGS, PocketGS) produce visually plausible models but without metric scale, making measurements impossible.

3. **No capture guidance**: No existing system provides real-time feedback on image quality, tracking state, and spatial coverage during capture, leading to frequent quality failures.

4. **Manual pipeline**: Research pipelines require manual steps (photo selection, COLMAP setup, training configuration) that non-experts cannot perform.

5. **No structure from a plain video**: Consumer 3DGS apps and research methods produce a radiance field but no explicit room structure (wall planes, wall lengths, doors) from which a floor plan or reliable measurement can be taken; where floor plans exist, they depend on LiDAR or dedicated hardware.

2.5 Proposed Solution

BuildScan 3D bridges these gaps by combining:

- **ARCore for metric scale**: The phone's visual-inertial odometry provides metric-scale poses that are refined via COLMAP SfM and aligned through a Sim3 transform, giving the 3D model real-world dimensions.

- **Live capture guidance**: Real-time image quality assessment (blur, exposure, motion speed) and spatial coverage visualization (voxel-based direction-aware coverage map) reduce capture failures.

- **Fully automated cloud pipeline**: A FastAPI server handles upload, dispatches processing to a RunPod GPU pod, monitors progress, and delivers the finished model—no manual steps.

- **State-of-the-art rendering**: Nerfstudio splatfacto 3DGS produces photorealistic models that render at real-time frame rates.

- **Structural layer**: wall planes, corners, wall lengths and doors extracted automatically from the splat model (`structure.py`, prototype), as the basis for floor plans and 1:1 measurement; completion in FYP-2.

2.6 Complex Computing Problem

The core complex computing problem is: *How can a sequence of monocular smartphone video frames, with noisy ARCore poses and sparse depth measurements, be automatically converted into a geometrically accurate, metrically scaled 3D Gaussian Splatting model that supports reliable real-world measurements?*

This involves:

- Robust Structure-from-Motion on casually captured, motion-blurred video with tracking gaps and silent ARCore re-anchoring events.
- Metric alignment between COLMAP's arbitrary-scale coordinate system and ARCore's metric frame via a Sim3 similarity transform with outlier rejection.
- Correct initialization of 3D Gaussians from multi-view stereo points to avoid degenerate geometry (zero-thickness splats, opacity collapse).
- Per-step GPU resource management with timeout watchdogs to prevent unbounded cloud billing.

2.7 Technical Challenges

1. **ARCore silent re-anchoring**: ARCore silently corrects its world frame during SLAM loop closures, causing pose discontinuities of up to 1 m / 28°. On our first room every pose before ∼60 s was off by about 1 m and 28°, so part of the room appeared twice. Solved by taking training poses from image-based COLMAP SfM, which does not depend on ARCore's world frame; the app now also logs ARCore anchors so that re-anchors are directly detectable. Genuine tracking losses are bridged with the IMU (`fallback.py`).

2. **Lens distortion**: ARCore's reported camera intrinsics lack lens distortion coefficients. COLMAP's self-calibration recovers radial distortion ($k_1 \approx 0.09$, $k_2 \approx -0.22$), which is essential for sub-pixel SfM accuracy.

3. **Degenerate Gaussian splats**: Our initial training recipe produced zero-thickness, half-transparent splats ("cotton texture") due to a flattening regularizer interacting pathologically with Adam's gradient normalization. Root-cause analysis (documented in `Training_Analysis.md`) led to adopting Nerfstudio splatfacto with measured-point initialization.

4. **Plain/textureless surfaces**: Rooms with plain white walls and ceilings cause tracking losses and produce inconsistent depth estimates. Solved via texture-aware IQA warnings and ceiling/floor capture guidance.

5. **GPU memory constraints**: The RTX 3070 (8 GB VRAM) limits splat count to approximately 3 million at full resolution. Addressed by training on a 24 GB cloud GPU (RTX 4090 via RunPod) and optimizing memory usage (images in pinned CPU RAM).

2.8 Research Findings and Design Decisions

Key research findings that shaped our design:

1. **SfM poses outperform ARCore poses for 3DGS**: Raw ARCore poses have a 2.32 px median reprojection error; bundle adjustment reduces it to 0.62 px (626k observations), but cannot remove ARCore's silent $\sim$1 m / 28° re-anchor or its missing lens model. Switching to image-based COLMAP poses with lens self-calibration raised the training-view fit from 21.9 dB (12–30 px misaligned) to $\sim$28 dB (2–3 px), raised held-out SSIM from 0.78 to 0.86 (0.91 on a second room), and cut badly placed surfaces from 7–23% to 3% (1% on the second room). For reference, the best model trained on the earlier poses was our own trainer with COLMAP depth (`gs_colmap_nc`, 22.67 dB / 0.782 held-out). *Decision*: Use COLMAP SfM for training poses; ARCore provides metric scale via Sim3 alignment.

2. **Dense initialization is critical**: PocketGS (2) demonstrated that dense multi-view stereo points (vs. sparse SfM points) significantly improve reconstruction quality (SSIM 0.834 vs. 0.791). *Decision*: Initialize from MVS-measured points voxel-downsampled at 1 cm.

3. **Regularization can be harmful**: Our flattening loss, depth loss, and pose refinement each degraded quality when applied together. The plain splatfacto recipe (photometric loss only, fixed poses) outperformed every combination. In a separate comparison on a different scene (`roomAttemptSadness2`), splatfacto on our preprocessed inputs reached 27.81 dB held-out (28.08 dB on training views), against 24.70 dB held-out for our own trainer with the corrected recipe. *Decision*: Use standard splatfacto; keep custom preprocessing.

4. **Gyro-aware frame selection improves sharpness**: Selecting frames by predicted motion blur (from IMU angular velocity and exposure time) instead of texture content improved PSNR by 0.26 dB. *Decision*: Implemented `select_sharp.py` using per-frame exposure metadata logged by the app.

2.9 Techniques Explored and Parked

Before the switch to image-based poses, the team tried a series of techniques to fix blurry, "cotton-like" models with floating splats. None of them fixed the underlying problem (ARCore's silent re-anchor and missing lens model; finding 1 above); the best model of that period reached only 22.67 dB / 0.782 on held-out frames. They are recorded here so that every technique attempted is visible. Numbers are from `docs/progress/`.

Table 2.2: Techniques explored and parked

Technique	**What we tried and measured**	**Outcome**
Own GPU bundle adjustment (`ba.py`)	Levenberg–Marquardt with the Schur complement accumulated on the GPU, ARCore as priors on relative and absolute pose. Median reprojection error 2.32 px $\rightarrow$ 0.61 px; trajectory length 0.998$\times$ ARCore.	Parked. Because it starts from ARCore and keeps ARCore priors, it kept the $\sim$1 m / 28° re-anchored segment. Replaced by COLMAP SfM.
Metric3D monocular depth	Single-image metric depth as a dense prior. Per-frame scale varied 0.85$\times$–1.50$\times$ against ARCore points; walls came out wavy. A smooth log-depth correction cut its error against MVS from 2.4% to 1.1%.	Parked. Its errors are regional and consistent across neighbouring frames, so they passed consistency checks and produced floating copies of furniture.
Own stereo matcher (`mvs.py`)	GPU plane-sweep MVS: 5 source views, 64 depth hypotheses, 7$\times$7 NCC, forward-backward consistency check. Only about 11% of pixels per frame were measured, because of plain walls.	Parked. COLMAP dense MVS is used for initialisation instead (v5, v6).
Depth supervision	Depth loss towards a fused surface that included single-image fill. Where stereo measured the true depth, the target was off by $>$5% on $\sim$30% of pixels and often lay behind the real surface.	Rejected. It fought the photometric loss and was one of the six causes of degenerate splats.

Table 2.2: Techniques explored and parked

Technique	**What we tried and measured**	**Outcome**
Floater pruning (`clean_splats.py`)	Delete splats contradicted by measured depth. Removed 1.4% of splats for -0.3 dB and no visible change; with filled depth as evidence it removed 9.8% but deleted real surfaces.	Rejected.
Median filtering	Hole-aware 5–9 px median on stereo depth: outliers ($>$25% error) 3.3% $\rightarrow$ 2.4–2.7%, but coverage was lost, and it does nothing for the Metric3D fill or for COLMAP depth.	Parked; kept only as an optional step in `colmap_mvs.py`.
Post-hoc flat-forcing (`planarize.py`)	Snapping walls and floor flat after training.	Rejected: it destroyed the glossy floor's reflections and crushed door frames. The structural layer is built next to the splat model instead.

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

During the proposal defense, the panel provided the following key feedback:

- Clarify the distinction between BuildScan 3D and existing consumer scanning apps (Polycam, Luma AI).
- Demonstrate metric accuracy with quantitative comparisons to ground truth measurements.
- Focus on measurement accuracy and the automated pipeline rather than feature-rich UI.
- Define clear acceptance criteria for reconstruction quality (PSNR, SSIM thresholds).

3.2 Proposal Action Register

No.	Feedback / Required Change	Action Taken	Status	Evidence
1	Differentiate from consumer apps	Added metric scale via ARCore Sim3, live IQA, and measurement capability	Done	Chapter 2, Table 2.1

2	Quantitative metric accuracy	Tape-measure comparison: mean error 4.8 mm (0.19%), worst case 0.29% against a 5% target	Done	Table 7.5
3	Define quality thresholds	Set acceptance: PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85, measurement error $\leq$ 5% of tape measure	Done	Section 1.9
4	Show automated pipeline	Built FastAPI server with RunPod cloud GPU integration	Done	Chapter 6, Section 6.6

3.3 Iteration Strategy

We follow a two-iteration strategy aligned with FYP-1 and FYP-2:

- **Iteration 1 (FYP-1, Weeks 1–8)**: Core model implementation—capture app, reconstruction pipeline, processing server, cloud GPU integration, validated on real recordings. Status at mid-evaluation: *core pipeline complete, refinement ongoing*. Deliverables: working Android APK, FastAPI server, RunPod integration, metric-accurate 3D reconstruction.
- **Iteration 2 (FYP-2, Weeks 9–16)**: Optimization, performance tuning, and feature enrichment. Deliverables: reduced processing time, pipeline robustness improvements, Three.js web viewer with measurement tools, structural layer and floor plans, field validation on multiple rooms.

3.4 Iteration 1 Plan

- **Objective**: Build the end-to-end pipeline from phone capture to 3D model.
- **Features**: ARCore capture with IQA + coverage overlay; COLMAP SfM + splatfacto training; FastAPI server with RunPod dispatch; upload UI in the Android app.
- **Deliverables**: Working Android APK, processing server, cloud GPU integration, reconstruction quality validation on two rooms.
- **Evidence**: Live demo of phone capture $\rightarrow$ upload $\rightarrow$ cloud processing $\rightarrow$ 3D model viewing.

3.5 Iteration 2 Plan

- **Objective**: Optimize processing time, improve pipeline robustness, complete the structural layer and add measurement tools.

- **Features**: Pipeline optimization (target $<$35 min on RTX 4090, projected, from $\sim$75 min measured on RTX 3070); improved handling of edge cases (mirrors, textureless walls, large rooms); Three.js/WebGPU splat viewer; point-to-point measurement tool that snaps to walls; structural layer completion (wall planes, corners, wall lengths, door detection, floor plans).

- **Deliverables**: Optimized pipeline with reduced processing time, web viewer with measurement tools, structural layer with floor-plan export, field validation report across $\geq$5 rooms.

- **Evidence**: Processing time benchmarks; measurement accuracy comparison with tape-measure ground truth across $\geq$5 rooms.

3.6 Work Breakdown and Responsibilities

3.7 Progress Against Plan

The core-pipeline milestones of Iteration 1 have been met; refinement is ongoing:

- **Week 1–2**: ARCore capture app with basic recording $\rightarrow$ *Completed*.

- **Week 3–4**: COLMAP SfM pipeline with ARCore Sim3 alignment $\rightarrow$ *Completed*. Also resolved root causes of poor reconstruction quality (tracking gaps, lens distortion, degenerate splats).

- **Week 5–6**: Live IQA, coverage overlay, quality improvements (gyro-aware frame selection, MVS initialization) $\rightarrow$ *Completed*.

- **Week 7–8**: Processing server, RunPod integration, upload UI, CI/CD $\rightarrow$ *Completed*.

- **Open items**: 3DGS refinement, 3D mesh creation, completion of the structural layer, and the web workspace.

Table 3.2: Work breakdown and ownership

Task	Owner	Status
ARCore capture app	Tayyab, Umair	Complete
Live IQA + coverage overlay	Tayyab (IQA), Umair and Tayyab (coverage)	Complete
Upload UI (ProcessingActivity)	Umair	Complete
Reconstruction pipeline (SfM)	Tayyab	Core complete; refinement ongoing
3DGS training & analysis	Tayyab; trainer fixes by Zaid	Refinement ongoing
FastAPI processing server	Umair	Complete
RunPod cloud GPU integration	Umair	Complete
CI/CD (GitHub Actions)	Zaid, Umair	Complete
Unit tests (pytest)	Zaid	Complete
Pipeline optimization (FYP-2)	Umair, Tayyab	Planned
Web viewer (FYP-2)	Zaid, Tayyab	Planned
3D mesh creation	Team	Planned; not yet scheduled
Structural layer (`structure.py`)	Tayyab	Prototype; completion FYP-2

3.8 Delays, Deviations and Recovery Plan

- **Deviation**: We initially built a custom training script (`train.py`) with depth supervision, flattening regularization, and pose refinement. After extensive analysis (documented in `Training_Analysis.md`), we found this produced worse results than standard splatfacto. We pivoted to using splatfacto for training and keeping our custom preprocessing. This analysis consumed approximately 2 weeks but resulted in a better-informed design decision.

- **Recovery**: The core pipeline is in place, but Iteration 1 is not finished: 3DGS refinement, 3D mesh creation, completion of the structural layer, and the web workspace remain open. The training analysis cost about two weeks. FYP-2 therefore starts with the one-command v6 pipeline and the processing-time reduction, and schedules the structural layer and web workspace in Iterations 2a–2b. 3D mesh creation

is in scope but not yet scheduled.

Chapter 4

Software Requirements

4.1 User Interactions / Use Cases

The system supports three primary user interactions:

1. **Capture**: The user opens the Android app, starts recording, walks through the room while receiving live quality feedback, stops recording, and taps "Upload & Process."

2. **Monitor**: The user watches the processing progress screen showing current step, percentage, elapsed time, and log output.

3. **View/Measure**: The user opens the finished 3D model in the web viewer and takes point-to-point measurements (FYP-2).

4.2 Use Case Diagram

The system involves three actors (Figure 4.1): the *End User* (captures and views), the *Processing Server* (manages jobs), and the *Cloud GPU* (runs reconstruction). The use cases are Record Scan, Upload Dataset, Monitor Progress, View Model, Take Measurement, and Configure Server. Each maps to code: Record Scan to `HelloArActivity` and `CaptureLogger`; Upload Dataset and Monitor Progress to `ProcessingActivity` and the `/upload` and `/status` endpoints of `server/main.py`; Configure Server to the app's settings menu (server URL) and `/config`; View Model and Take Measurement to the desktop viewer today and the web workspace in FYP-2.

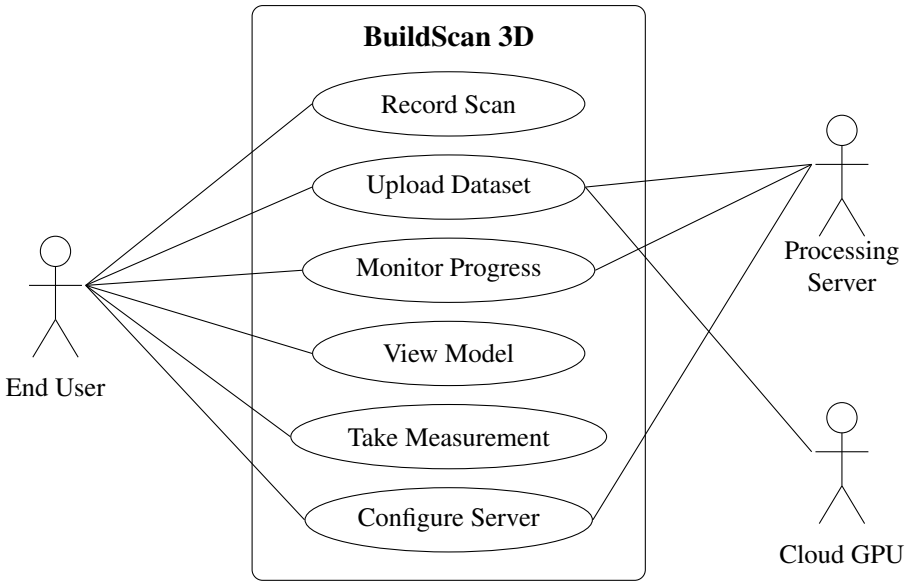


Figure 4.1: Use case diagram. The End User is the primary actor; the Processing Server (`server/main.py`) and the Cloud GPU (RunPod pod, or the local GPU) are secondary actors. Take Measurement currently runs in the desktop viewer; the web version is FYP-2 work.

4.3 Detailed Use Cases

UC-1: Record Indoor Scan

- **Actor**: End User
- **Precondition**: App installed on ARCore-compatible Android device
- **Main flow**: User opens app → taps Record → walks through room → receives live IQA feedback (blur, speed, darkness, texture warnings) and coverage overlay (red/yellow/green dots) → taps Stop → sees quality summary
- **Postcondition**: Dataset saved locally (video, poses, depth, quality, meta)

UC-2: Upload and Process

- **Actor**: End User
- **Precondition**: Recording completed; server URL configured
- **Main flow**: User taps "Upload & Process" → app zips depth folder, uploads all files → server creates job, dispatches to GPU → app polls progress every 3 s showing step, percentage, logs → completion or failure reported

- **Postcondition**: 3D model (splat.ply) available for download
- **Exception**: Upload fails → retry button; pipeline step fails → error message with step name

4.4 Functional Requirements

4.4.1 Mobile Capture Module

- **FR-M1**: The app shall record video at 30 FPS alongside per-frame camera poses (position + quaternion + intrinsics) in `poses.csv`.
- **FR-M2**: The app shall log ARCore tracking state and failure reasons in `tracking.csv`.
- **FR-M3**: The app shall save ARCore feature points in `points.csv`.
- **FR-M4**: The app shall save raw depth maps at 5 Hz in `depth/` with confidence maps.
- **FR-M5**: The app shall compute and display live IQA metrics: blur, brightness, motion speed, texture level.
- **FR-M6**: The app shall display a coverage overlay showing scanned (green), partially scanned (yellow), and unscanned (red) surfaces.
- **FR-M7**: The app shall upload the complete dataset to the processing server via multipart HTTP POST.
- **FR-M8**: The app shall allow the user to configure the server URL via the settings menu.
- **FR-M9**: The app shall display upload and processing progress with step name, percentage, elapsed time, and recent log lines.

4.4.2 Processing Server Module

- **FR-S1**: The server shall accept multipart dataset uploads (video, poses, tracking, points, quality, meta, depth.zip).
- **FR-S2**: The server shall create a unique job ID for each upload and track its status.
- **FR-S3**: The server shall run the reconstruction pipeline with per-step timeout watchdogs.

- **FR-S4**: The server shall support two processing modes: local GPU and RunPod cloud GPU.
- **FR-S5**: The server shall expose a progress-polling endpoint returning current step, percentage, elapsed time, and recent log lines.
- **FR-S6**: The server shall provide a download endpoint for the finished `splat.ply`.
- **FR-S7**: In RunPod mode, the server shall always terminate the GPU pod after processing (even on failure).

4.4.3 Reconstruction Pipeline Module

- **FR-P1**: The pipeline shall extract keyframes from the video based on gyro-aware sharpness prediction.
- **FR-P2**: The pipeline shall run COLMAP SfM with SIFT features, lens self-calibration, and global mapping.
- **FR-P3**: The pipeline shall compute a Sim3 alignment from COLMAP's coordinate system to ARCore's metric frame.
- **FR-P4**: The pipeline shall train a splatfacto 3DGS model and export `splat.ply`.
- **FR-P5**: The pipeline shall bridge ARCore tracking gaps using IMU data (`fallback.py`).

4.5 Non-Functional Requirements

4.5.1 Reliability

- **NFR-R1**: The cloud GPU pod shall be terminated within 60 s of job completion or failure (prevent billing leaks).
- **NFR-R2**: Each pipeline step shall have a timeout watchdog; if exceeded, the step is killed and the job marked failed.

4.5.2 Usability

- **NFR-U1**: The capture app shall provide real-time visual feedback without requiring the user to understand 3D reconstruction concepts.
- **NFR-U2**: Processing progress shall be displayed with a progress bar, step name, and elapsed time.

4.5.3 Performance

- **NFR-P1**: End-to-end processing time (upload to exported `splat.ply`) shall be under 35 minutes on an NVIDIA RTX 4090 cloud GPU. This is an FYP-2 target and a projection: the only measured time is $\sim$75 minutes on the development RTX 3070 (matching 23, mapping 6, training 45 min).
- **NFR-P2**: The capture app shall maintain 30 FPS camera preview during recording (no frame drops from logging overhead).
- **NFR-P3**: The app's recording fast-path shall skip all non-essential rendering (planes, virtual objects, PBR lighting) to minimize GL thread load.

4.5.4 Security

- **NFR-S1**: The RunPod API key shall be loaded from environment variables or a `.env` file, never hardcoded.
- **NFR-S2**: The `.env` file shall be excluded from version control via `.gitignore`.

4.6 Acceptance Criteria

4.7 Requirements Traceability

Table 4.1: Acceptance criteria for key requirements

Req.	Criterion	Measurement
FR-M1	Pose logged for every video frame	Row count in poses.csv equals frame count in video
FR-P3	Metric accuracy within 5%	Distances in the model vs. tape measure on rigid objects; measured mean 0.19%, worst 0.29% (Table 7.5)
FR-P4	Reconstruction quality	Held-out PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85
FR-S7	Pod always terminated	Server logs confirm `terminate_pod()` in all paths
NFR-P1	Processing under 35 min on RTX 4090	Wall-clock time from upload to exported `splat.ply` on an RTX 4090 pod. *Outstanding: not yet measured* (75 min measured on RTX 3070)
NFR-P2	30 FPS during recording	No frame drops in logcat during 3-minute recording

Table 4.2: Requirements traceability matrix (selected)

Req.	Module	Design	Implementation	Test
FR-M1	Mobile Capture	CaptureLogger	CaptureLogger.java	TC-1
FR-M5	Mobile Capture	QualityMonitor	QualityMonitor.java	TC-2
FR-M6	Mobile Capture	CoverageMap	CoverageMap.java	TC-3
FR-S4	Processing Server	RunPodClient	runpod_client.py	TC-7
FR-P2	Recon. Pipeline	COLMAP SfM	sfm_colmap.py	TC-5
FR-P3	Recon. Pipeline	Sim3 Alignment	sfm_colmap.py	TC-6
FR-P5	Recon. Pipeline	IMU Fallback	fallback.py	TC-4

Chapter 5

System Design

5.1 System Architecture

BuildScan 3D uses a three-tier client-server architecture:

1. **Client tier**: Android application (ARCore capture, upload, progress display).
2. **Server tier**: FastAPI backend (job management, pipeline orchestration, RunPod dispatch).
3. **Compute tier**: GPU processing (local or RunPod on-demand pod running COLMAP + Nerfstudio).

The data flow is strictly sequential: Capture → Upload → Process → Download. The server acts as an orchestrator that manages the GPU lifecycle and provides progress updates to the client via polling. Figure 5.1 shows the components of each tier and how they talk to each other. The server runs in one of four processing modes set through `/config`: `dry_run` (the default: uploads are saved, nothing is processed), `local`, `runpod` and `remote` (a fixed GPU host over SSH).

5.2 Architectural Components

Android App (`googleARCore/samples/hello_ar_java/`)

- `HelloArActivity`: Main AR camera activity with recording controls, settings menu, and quality overlay.
- `CaptureLogger`: Background I/O thread writing poses, tracking, points, quality, depth, and metadata files.

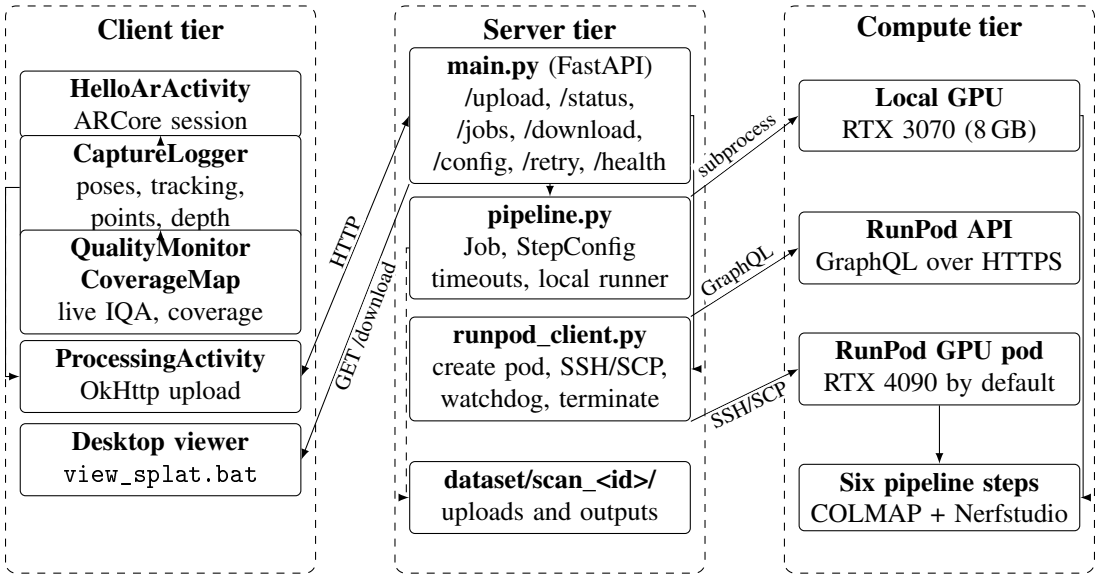


Figure 5.1: System architecture. Client: Android app (`googleARCore/samples/hello_ar_java/`) and the desktop viewer. Server: `server/main.py`, `pipeline.py`, `runpod_client.py`. Compute: the local GPU, or a RunPod pod reached through the RunPod GraphQL API and SSH/SCP. Both compute paths run the same six steps; `test_server_pipeline.py` checks that the two command lists match.

- `QualityMonitor`: Analyzes camera frames for blur, brightness, motion speed, and texture.
- `CoverageMap`: Voxel-based (10 cm) coverage tracker with 8-azimuth $\times$ 2-elevation direction awareness.
- `CoverageOverlayView`: Renders red/yellow/green dots over the camera view.
- `ProcessingActivity`: Upload + progress UI with OkHttp multipart client.

Processing Server (`server/`)

- `main.py`: FastAPI endpoints—`/upload`, `/status/{job_id}`, `/jobs`, `/logs/{job_id}`, `/retry/{job_id}`, `/jobs/{job_id}/cancel`, `/download/{job_id}`, `/config`, `/gpus`, `/datasets`, `/reprocess`, `/run-on-pod`, `/health`.
- `pipeline.py`: Local pipeline runner with `Job` dataclass, `StepConfig` per-step timeouts, and subprocess execution with streaming stdout.
- `runpod_client.py`: RunPod API client (GraphQL pod creation, SSH/SCP helpers, remote pipeline execution, pod watchdog, pod termination in `finally` blocks).

Reconstruction Pipeline (`src/arcore3dgs/`)

- `fallback.py`: IMU-bridged tracking-gap recovery.
- `prepare.py`: Keyframe extraction from video (sharpest frame by Laplacian variance in each window of 8 frames).
- `select_sharp.py`: Gyro-aware keyframe selection (least predicted motion blur per 0.2 s window). Used for the v4–v6 models; not yet part of the one-command pipeline.
- `sfm_colmap.py`: COLMAP SIFT matching + global mapper + lens self-calibration + Sim3 alignment to ARCore metric frame.
- `export_nerfstudio.py`: Convert COLMAP output to Nerfstudio data format.
- Nerfstudio `ns-train splatfacto`: 3DGS training (30k iterations).
- Nerfstudio `ns-export`: Export `splat.ply`.

5.3 Interfaces and Dependencies

Table 5.1: Key interfaces between components

From	To	Interface
Android App	Server	HTTP multipart POST (`/upload`)
Android App	Server	HTTP GET polling (`/status/{id}`)
Server	RunPod API	GraphQL over HTTPS
Server	GPU Pod	SSH/SCP over TCP
Pipeline steps	Next step	Filesystem (dataset directory)
`sfm_colmap.py`	splatfacto	`transforms.json` + images

External dependencies:

- Google ARCore SDK (Android)
- COLMAP 4.2 (SfM/MVS)
- Nerfstudio 1.1.5 (3DGS training)
- OkHttp 4.12.0 (Android HTTP client)

- FastAPI + Uvicorn (Python web framework)
- RunPod API (cloud GPU provisioning)

5.4 Architecture Decisions and Rationale

1. **COLMAP SfM over ARCore poses for training**: Raw ARCore poses have a 2.32 px median reprojection error (slow drift); bundle adjustment reduces this to 0.62 px, but ARCore also re-anchors silently (every pose before $\sim$60 s was off by $\sim$1 m and 28°) and reports no lens distortion. COLMAP's SfM with self-calibrated lens distortion ($k_1 \approx 0.09$, $k_2 \approx -0.22$) removes both. On the same recording, training-view fit rose from 21.9 dB to $\sim$28 dB, held-out SSIM from 0.78 to 0.86, and badly placed surfaces fell from 7–23% to 3%.
2. **Splatfacto over custom trainer**: After extensive analysis showing our custom training recipe produced degenerate splats, we adopted Nerfstudio's battle-tested splatfacto implementation. On a separate scene (`roomAttemptSadness2`), splatfacto reached 27.81 dB held-out against 24.70 dB for our corrected custom trainer on the same inputs. Our value-add is in the preprocessing (ARCore integration, metric alignment, IMU fallback, quality guidance).
3. **RunPod on-demand over serverless**: On-demand pods allow full SSH control of the pipeline, suit long-running jobs (about 75 min end-to-end measured on an RTX 3070), and support arbitrary software installation. Serverless endpoints have cold-start latency and timeout limits that conflict with COLMAP's variable runtime.
4. **HTTP polling over WebSockets**: Simpler implementation, works across firewalls/proxies, and 3-second polling interval is sufficient for a pipeline that takes minutes.

5.5 Design Evolution

1. **Proposal → Mid**: Added live IQA and coverage guidance (not in original proposal). Changed from ARCore-pose-based training to COLMAP SfM-based training after discovering ARCore's pose accuracy limitations. On the way, the team's own GPU bundle adjustment, Metric3D depth, own stereo matcher, depth supervision, floater pruning, median filtering and post-hoc flat-forcing were tried and parked (Section 2.9).
2. **Custom trainer → splatfacto**: After two weeks of analysis documenting six root causes of degenerate splat geometry, we switched training to Nerfstudio splatfacto. The custom preprocessing pipeline was retained and validated.

3. **Local-only → local + cloud**: Added RunPod cloud GPU support to enable processing on machines without a GPU, and to support faster GPUs (RTX 4090 by default) than the development machine (RTX 3070, 8 GB).

5.6 Domain Model

The core domain entities are:

- **Scan**: A recording session producing a dataset (video + sensor files).
- **Dataset**: The collection of files from one scan (video, poses, depth, quality, meta).
- **Job**: A processing task with status (queued, processing, completed, failed), progress, and output path. Its life cycle is shown in Figure 5.2.
- **Model**: The output 3D Gaussian Splatting model (`splat.ply`).
- **Measurement**: A point-to-point distance in the metric model (FYP-2).

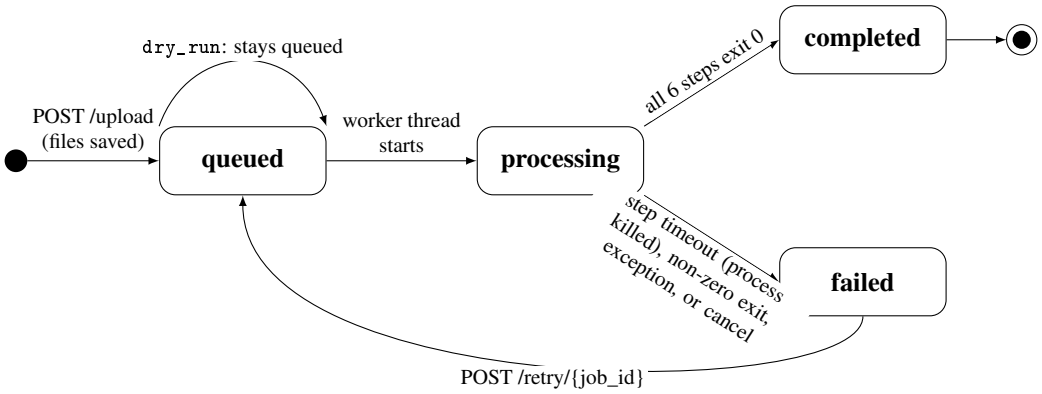


Figure 5.2: Job state machine (`JobStatus` in `server/pipeline.py`). A timeout is enforced per step by the watchdog loop in `run_pipeline()`, which kills the subprocess and marks the job failed; cancel is `POST /jobs/{job_id}/cancel`. In RunPod mode a separate pod watchdog terminates the pod after `MAX_POD_LIFETIME` (default 7200 s). An `uploading` value is defined in `JobStatus` but no code path sets it.

5.7 Data Design

Capture data (per scan):

- `dataset.mp4`: H.264 video, 1920×1080, 30 FPS.

- `poses.csv`: timestamp, tx, ty, tz, qx, qy, qz, qw, fx, fy, cx, cy (per frame).
- `tracking.csv`: timestamp, tracking_state, failure_reason (per frame).
- `points.csv`: timestamp, point_id, x, y, z, confidence (ARCore feature points).
- `quality.csv`: timestamp, sharpness, brightness, motion_speed, texture_level (per analyzed frame).
- `depth/`: uint16 depth maps (mm) + uint8 confidence maps at 5 Hz.
- `meta.json`: device model, camera config, resolution, ARCore version.

Pipeline intermediate data:

- `poses_fused.csv`, `fallback_report.json`: poses with tracking gaps bridged by the IMU.
- `work/images/`, `work/frames.json`: extracted keyframes (JPEG) and their ARCore poses.
- `work_sfm/`: COLMAP database, sparse model, undistorted images, Sim3-aligned `frames.json`, `init_points.ply`.
- `work_sfm/transforms.json`: Nerfstudio-format camera transforms.
- `work_sfm/gs_sfm/splat.ply`: Final 3DGS model.

Figure 5.3 traces the capture files through the six pipeline steps. `quality.csv`, `meta.json` and `depth/` are uploaded and stored with the dataset, but the current six steps do not read them (keyframe extraction runs with `-depth 0`).

5.8 Process / Behavior Design

The end-to-end pipeline follows a sequential six-step process:

1. **Fallback** (2 min timeout): Read the gyroscope and accelerometer samples stored in `dataset.mp4`, find tracking gaps in `tracking.csv`, and bridge them with IMU integration anchored at both ends of each gap.
2. **Prepare** (5 min timeout): Decode video, keep the sharpest frame (Laplacian variance) in each window of 8 frames, extract to JPEG (`prepare.py`). The gyro-aware selection of `select_sharp.py` that produced v6 is not yet in this step.

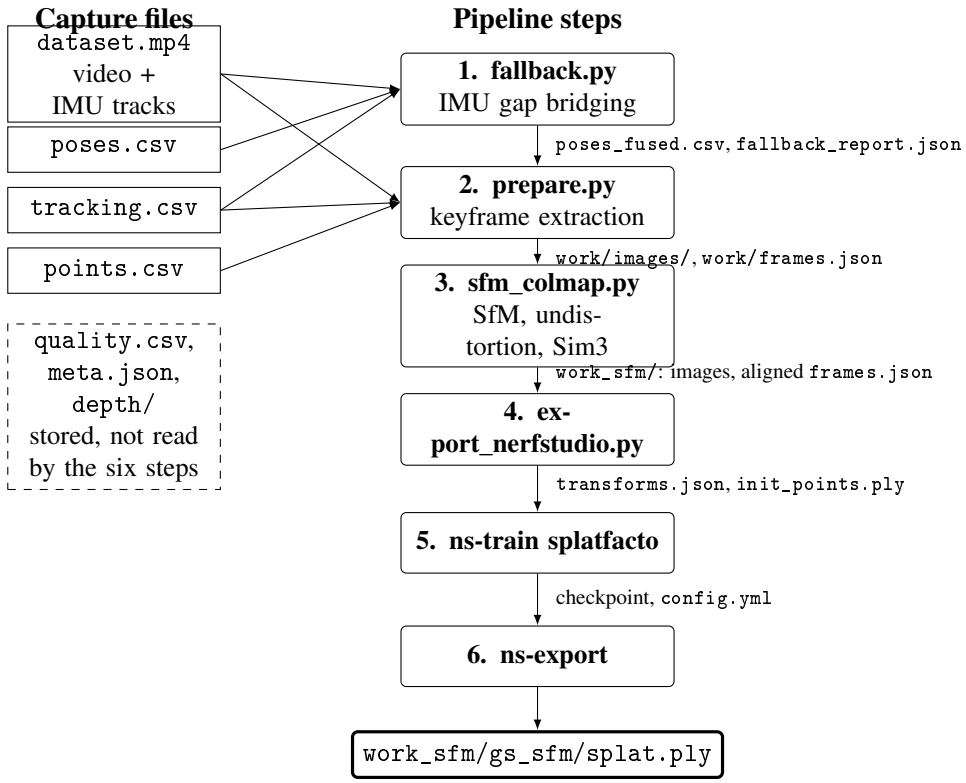


Figure 5.3: Data flow from the capture files to the model, as run by `server/pipeline.py` (and, step for step, on a RunPod pod). Steps 1–2 read the capture through `common.Capture`; `fallback.py` reads the IMU samples from the MP4's own data tracks (`imu.py`).

3. **SfM** (40 min timeout): COLMAP feature extraction (SIFT), exhaustive matching, global mapper, lens self-calibration, image undistortion, Sim3 alignment to ARCore metric frame.

4. **Export** (2 min timeout): Convert COLMAP output to Nerfstudio `transforms.json` format.

5. **Train** (60 min timeout): Nerfstudio splatfacto training (30k iterations) with bilateral grid exposure compensation, scale regularization, and fixed poses.

6. **Export Model** (5 min timeout): Export trained model to `splat.ply`.

Each step is executed as a subprocess with streaming stdout captured for progress reporting. If a step exceeds its timeout, the process is killed and the job is marked failed. In RunPod mode, whenever a pod has been created it is terminated in a `finally` block regardless of success or failure, and a watchdog thread terminates it after `MAX_POD_LIFETIME` (default 7200 s). Figure 5.4 shows the upload → process → download interaction in RunPod mode.

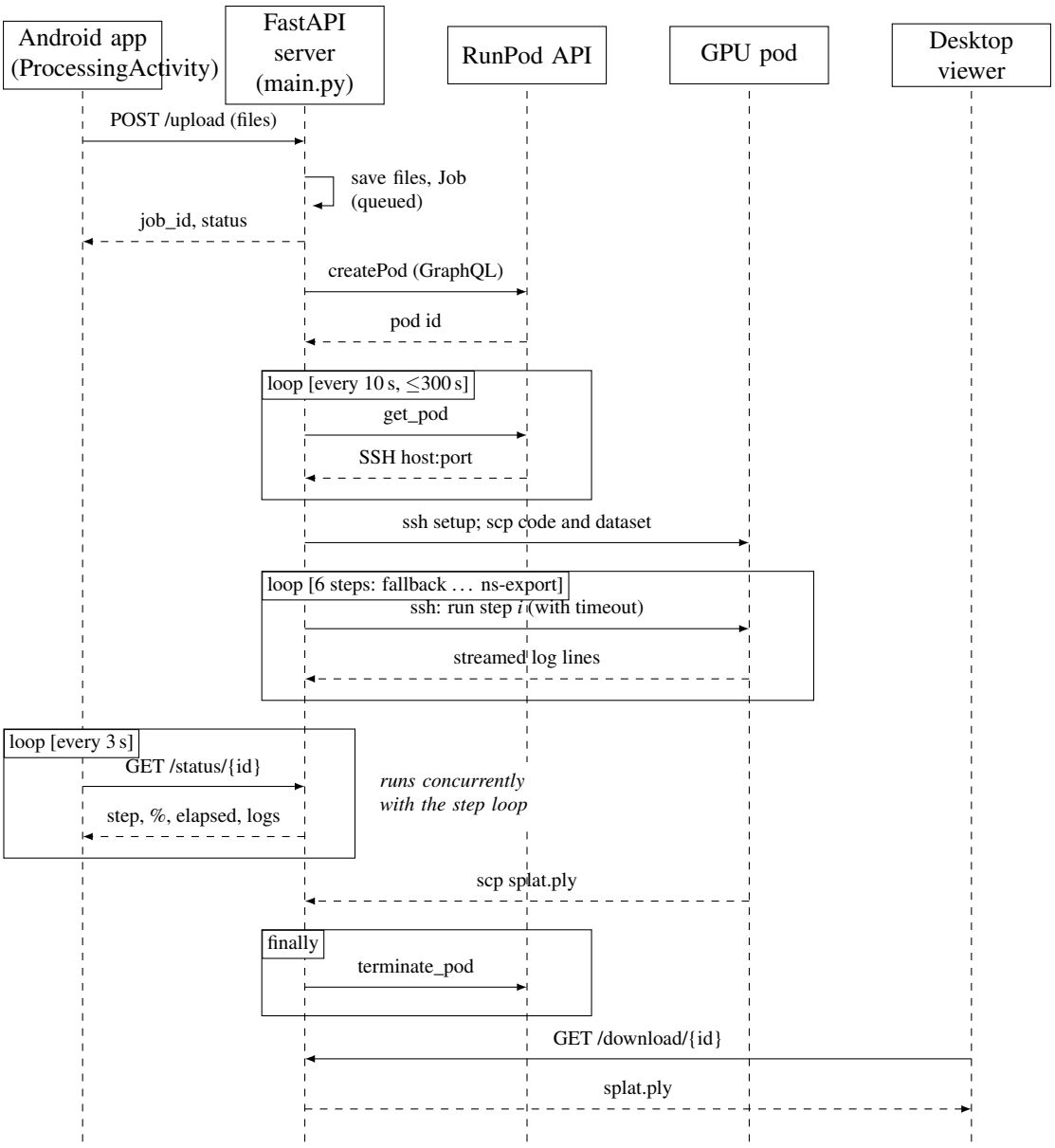


Figure 5.4: Sequence diagram for upload → process → download in RunPod mode (`ProcessingActivity.java`, `server/main.py`, `run_pipeline_on_runpod()` in `server/runpod_client.py`). The app polls every 3 s (5 s after an error) and stops when the status is `completed` or `failed`. The app does not download the model; it is fetched with `GET /download` by the desktop viewer.

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

Build the complete end-to-end pipeline: phone capture → automated upload → cloud GPU reconstruction → metric-scale 3D model. Validate reconstruction quality and metric accuracy on real indoor scans.

6.2 Selected Use Cases / Features

- UC-1: Record Indoor Scan (with live IQA and coverage guidance)
- UC-2: Upload and Process (with progress tracking)
- UC-3: View finished model (via local viewer; web viewer deferred to FYP-2)

6.3 Design

The Iteration 1 design focused on three subsystems:

1. **Capture subsystem**: ARCore session → CaptureLogger (background I/O) → QualityMonitor (frame analysis at 6 FPS) → CoverageMap (depth → voxels → overlay).
2. **Upload subsystem**: OkHttp multipart POST → FastAPI `/upload` → Job creation → Pipeline dispatch (local or RunPod).
3. **Pipeline subsystem**: Six sequential steps (fallback → prepare → sfm_colmap → export_nerfstudio → ns-train → ns-export), each with timeout watchdog.

6.4 Implementation

Implementation was carried out in Java (Android app) and Python (server and pipeline). Key implementation milestones:

1. **Weeks 1–2**: Extended the ARCore hello_ar_java sample with `CaptureLogger` for sensor data recording, verified pose-to-video synchronization ($r = 0.76$–0.79 at 0.00 s offset).
2. **Weeks 3–4**: Built the COLMAP SfM pipeline (`sfm_colmap.py`), resolved coordinate alignment issues (OpenGL → OpenCV), implemented Sim3 metric alignment, debugged lens distortion.
3. **Weeks 5–6**: Diagnosed and fixed degenerate splat geometry (6 root causes documented in `Training_Analysis.md`), added gyro-aware frame selection, live IQA, and coverage overlay.
4. **Weeks 7–8**: Built the FastAPI server, RunPod cloud GPU integration, Android upload UI (`ProcessingActivity`), CI/CD pipeline.

6.5 Completed Features / Module

Android Capture App (implemented):

- 30 FPS video recording with per-frame pose, tracking state, feature points, depth, quality metrics, and device metadata.
- Live IQA with colour-coded banner: tracking lost (red), moving too fast (orange), blurry (yellow), too dark (yellow), low texture (yellow).
- Haptic feedback (vibration) on quality degradation during recording.
- Coverage overlay: red (1 direction), yellow (2–3), green (4+) dots at 2 Hz.
- Per-frame exposure time, rolling-shutter readout, and anchor correction logging.
- End-of-recording quality summary dialog.
- Upload & Process button launching `ProcessingActivity`.
- Configurable server URL via settings menu (persisted in SharedPreferences).
- Performance optimization: recording fast-path skips all demo rendering.

Processing Server (implemented):

- FastAPI with endpoints: `POST /upload`, `GET /status/{id}`, `GET /jobs`, `GET /download/{id}`, `GET/POST /config`, `GET /health`.
- Local and RunPod processing modes switchable at runtime.
- Per-step timeout watchdogs (fallback 2 min, prepare 5 min, SfM 40 min, export 2 min, train 60 min, export_splat 5 min).
- RunPod integration: pod creation, SSH readiness polling, dependency installation, SCP upload/download, guaranteed pod termination.
- `.env` file support for API key management.

Reconstruction Pipeline (core complete, refinement ongoing):

- IMU-bridged tracking-gap recovery (`fallback.py`).
- Keyframe extraction (`prepare.py`, sharpest frame per window) in the one-command pipeline; gyro-aware sharp frame selection (`select_sharp.py`, using per-frame exposure metadata) validated for v4–v6 but not yet in the one-command pipeline.
- COLMAP SfM with SIFT features, exhaustive matching, global mapper, lens self-calibration (radial distortion k_1, k_2), and undistortion.
- Sim3 alignment to ARCore metric frame (tape-measure error: mean 0.19%, worst case 0.29%).
- Nerfstudio splatfacto training with bilateral grid exposure compensation.
- Model export to `splat.ply`.

CI/CD (implemented):

- GitHub Actions CI: Python lint (ruff) + unit tests (pytest, 3 Python versions), web UI build, Android APK build, repo hygiene (file size, secrets, workflow lint).
- GitHub Actions CD: on push to master, full CI gate $\rightarrow$ web bundle + APK artifacts; on version tag, GitHub Release.
- Pull request template with area checkboxes and CI checklist.

6.6 Technical Implementation Details

Pose-to-video synchronization (`CaptureLogger.java`): ARCore's recording API writes video and poses from the same frames, but poses are only logged during `TRACKING` state, creating gaps. The pipeline interpolates poses at each frame's PTS (linear position, slerp rotation) and skips frames inside tracking gaps > 0.12 s.

Sim3 metric alignment (`sfm_colmap.py`): COLMAP's global mapper produces an arbitrary-scale coordinate system. We compute a 7-DOF similarity transform (rotation + translation + scale) from matched COLMAP and ARCore camera positions using RANSAC (2,000 four-point samples, 10 cm inlier threshold) followed by a Umeyama refit on the inliers; the inlier median residual is about 2.5 cm. Because ARCore re-anchors silently, only its largest self-consistent segment ends up as inliers. Frames that jump more than 30 cm from their temporal neighbours are dropped as SfM mis-registrations. The resulting metric scale is validated against a tape measure (Table 7.5).

Recording fast-path (`HelloArActivity.java`): During recording, the `onDrawFrame()` method exits early after camera feed rendering and quality overlay (at 2 Hz). All plane detection rendering, virtual object placement, PBR lighting, cubemap filtering, and tap handling are skipped, reducing GL thread load significantly.

Featureless-tracking fallback (`imu.py`, `fallback.py`, `relocalize_pnp.py`): On plain walls ARCore loses tracking, and after re-tracking it sometimes continues in a shifted or rotated world frame. The fallback reads the gyroscope and accelerometer (201 Hz) stored in ARCore's own recording, self-calibrates the IMU-to-camera rotation and the sensor biases, and bridges each gap: rotation by gyro integration, position by accelerometer integration with a minimal cubic correction that matches position and velocity at both ends. A heading mismatch at re-tracking reveals a new map frame, and the post-gap segment is re-aligned to the map by matching its keyframes against it (PnP). This is the team's main novelty claim. Validation on gaps cut out of tracked data:

Table 6.1: IMU fallback on simulated tracking gaps (position error, median / max)

Simulated gap	IMU fallback	Interpolation
2 s	0.6 / 2.8 cm	5.0 / 12.3 cm
5 s	1.5 / 3.4 cm	20.6 / 52.4 cm
9 s	**3.8** / 8.5 cm	**30.6** / 75.9 cm

On a real recording, all three heading jumps were detected, and re-alignment cut the

error of post-gap frames against earlier frames from **49.7 px to 8.2 px**. Limits: IMU-only position is reliable only when both ends of a gap are anchored, and re-alignment needs the user to return to an already mapped area (the app now says so during tracking loss).

Structural layer (`structure.py`, prototype): The photoreal splat model is not modified. Instead the room shell is rebuilt next to it: the floor plane from the distribution of splat heights; the outermost wall on each side of the two dominant wall directions, each refined individually; corners as intersections of neighbouring walls, giving the floor polygon; per-wall textures as the median over the best views, which removes moving reflections; and openings where the cameras consistently see through a wall plane, classified as doors by their size and position. On the first room it found wall planes of 6.65–6.68 × 4.45–4.48 m and **detected 5 doors automatically**. Output is `structure.json` (planes, corners, dimensions, openings), the basis for automatic floor plans and 1:1 measurement. Ceiling-height detection is not yet reliable and is FYP-2 work.

RunPod lifecycle (`runpod_client.py`): The `run_pipeline_on_runpod()` function wraps the entire pod lifecycle in a try/finally block. Whenever a pod was created, the `finally` clause calls `terminate_pod()`, and a watchdog thread enforces a hard time limit, so that that even exceptions, timeouts, or SSH failures do not leave a pod running and accumulating charges.

6.7 Algorithms and Technical Decisions

1. **Gyro-aware frame selection**: For each 0.2 s window, compute predicted motion blur as blur_px $= \omega \times t_{\text{exp}} \times f$, where ω is IMU angular velocity (rad/s), t_{exp} is exposure time, and f is focal length in pixels. Select the frame with minimum predicted blur. This outperforms Laplacian-based sharpness ($r = 0.20$ correlation with true blur).

2. **Coverage map**: Discretize 3D space into 10 cm voxels. For each voxel, track from how many of 16 directions (8 azimuth × 2 elevation) it has been observed. Display as dots: red (1 direction), yellow (2–3), green (4+). Cap processing at 5,000 voxels and 2,000 drawn dots for performance.

3. **Degenerate splat diagnosis**: Computed per-model statistics (thickness/width median, opacity histogram, splat count) and compared against reference implementations. Identified flattening regularizer × Adam interaction as the primary cause of zero-thickness splats.

6.8 Tools, Frameworks and Technologies

Table 6.2: Tools and technologies used

Tool	Version	Purpose
Android Studio	Ladybug	Android app development
ARCore SDK	1.44+	Camera pose tracking, depth
OkHttp	4.12.0	HTTP client for upload
Python	3.10–3.12	Server and pipeline
FastAPI	0.110+	REST API server
COLMAP	4.2	Structure from Motion
Nerfstudio	1.1.5	3DGS training (splatfacto)
PyTorch	2.1+	GPU computation
OpenCV	4.9+	Image processing
GitHub Actions	–	CI/CD
RunPod API	–	Cloud GPU provisioning
ruff	0.6+	Python linting
pytest	8+	Unit testing

6.9 Demonstration Evidence

The following can be demonstrated live:

1. **Android app**: Record a room scan with live IQA feedback and coverage overlay.
2. **Upload flow**: Tap "Upload & Process," watch progress screen update.
3. **Server**: Start server with `scripts/start_server.bat`, show API docs at `/docs`.
4. **3D model**: View the reconstructed `splat.ply` in the local viewer (`scripts/view_splat.bat`).
5. **Metric accuracy**: Measure the table and a wall in the model and compare with the tape-measure values in Table 7.5.
6. **CI/CD**: Show GitHub Actions passing on the repository.

6.10 Contributions and Novelty

The contributions so far, each backed by measured evidence in this report:

1. **Measured evidence of why commodity ARCore poses fail for 3DGS**—a silent $\sim$1 m / 28° re-anchor and a missing lens model ($k_1 \approx 0.09$, $k_2 \approx -0.22$)—and a working fix: image-based poses with ARCore kept for scale and gravity (Table 7.4).
2. **A featureless-tracking IMU fallback**, validated on simulated gaps: 3.8 cm error against 30.6 cm for interpolation over 9 s, and post-gap re-alignment from 49.7 px to 8.2 px (Table 6.1).
3. **Live capture-quality guidance and a coverage overlay in the app**: warnings for blur, fast motion, darkness and plain surfaces, a vibration alert, and a live overlay of unmapped areas.
4. **Gyro-aware frame selection that measurably improves quality**: +0.26 dB / +0.009 SSIM, the largest single gain on the way to v6 (Table 7.3).
5. **Millimetre-level metric accuracy from a phone video, without LiDAR**: mean 4.8 mm (0.19%), worst case 0.29% against a tape measure (Table 7.5).

6.11 Iteration Outcome

Iteration 1 status: **core pipeline complete, refinement ongoing**. Results so far:

- Reconstruction quality clears the 25 dB PSNR acceptance bar: the current best model (v6) scores 26.52 dB PSNR / 0.8886 SSIM on 42 held-out frames that no model was trained on.
- Metric accuracy: tape-measure mean error 4.8 mm (0.19%), worst case 0.29%, against a 5% target.
- Processing time is approximately 75 minutes measured on the local RTX 3070 (matching 23, mapping 6, training 45 min). The FYP-2 target is under 35 minutes on an RTX 4090 (RunPod); this is a projection and has not yet been measured.
- The app provides live quality guidance and coverage visualization.
- The server supports both local and cloud GPU processing with automatic lifecycle management.

Open items (the FYP-2 schedule for all but mesh creation is in the Remaining FYP Plan chapter):

- **3DGS refinement**: make the v6 recipe the one-command pipeline, validate it on the second room, and model the remaining motion blur.

- **3D mesh creation**: in scope as planned work; not yet scheduled.
- **Structural layer completion**: wall-to-wall dimensions, doors and openings, ceiling-height detection, floor-plan export.
- **Web workspace**: browser viewer with 1:1 measurement.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

Our testing strategy combines automated unit/integration tests (CI-enforced) with manual system-level validation on real indoor scans. The rationale is that unit tests verify algorithmic correctness (coordinate transforms, file parsing, API contracts), while end-to-end validation on real data verifies reconstruction quality and metric accuracy—properties that cannot be unit-tested.

- **Unit tests**: Python pipeline modules tested with pytest on small synthetic capture folders (Python 3.10, 3.11, 3.12; no GPU, network or phone needed). Run by GitHub Actions CI on pushes to non-master branches, on pull requests, and from the CD workflow.
- **Lint**: ruff in CI; real errors fail the run, unused-code findings are informational only.
- **Integration tests**: Server endpoint tests through the FastAPI test client with the job starters replaced by recorders (upload, status, download, config), and RunPod client tests with the network faked. No test runs against a live RunPod pod.
- **Android**: There are **no Android (Java) unit tests**. The capture app is checked by using it on a phone, and its packaging by the APK/manifest checker tests.
- **System tests**: Manual runs of the reconstruction pipeline on real recordings, followed by held-out evaluation and tape-measure comparison. The full app upload $\rightarrow$ server $\rightarrow$ RunPod $\rightarrow$ download path has not yet been run as a recorded test.
- **Regression tests**: Each pipeline change is compared on the same two reference recordings (the first room, `newRoomAttemptSadness`, and a second room, `roomAttemptSadness2`)

to detect quality regressions. This comparison is run by hand with the evaluation scripts, not in CI.

7.2 Test Levels and Techniques

Table 7.1: Test levels and techniques

Level	Scope	Technique	Automation
Unit	Python pipeline modules	pytest on synthetic captures	CI (GitHub Actions)
Static analysis	All Python code	ruff; every file compiles	CI (GitHub Actions)
Integration	Server API, RunPod client	FastAPI test client; faked network	CI (GitHub Actions)
Packaging	Android APK and manifest	APK checker tests	CI (GitHub Actions)
Device	Android capture app	On-device use	Manual; no Java unit tests
System	Capture to model	Pipeline runs on real recordings	Manual
Acceptance	Quality and metric accuracy	Held-out frames; tape measure	Manual

7.3 Test Cases

Table 7.2 summarises the test cases; Appendix C gives the input, expected result and evidence for each, and lists what each pytest file covers. "Pass (CI)" refers to a full local run of the suite on the current `master` head (commit `0d78466`, 22 October 2026): 194 passed, 1 skipped, ruff clean; cross-version evidence (Python 3.10–3.12) comes from GitHub Actions CI run #6 (12 October 2026, commit `c3bfcc9`). "Pass (manual)" means a recorded run whose numbers are in `docs/progress/`.

Table 7.2: Test cases and their status

ID	Description	Evidence	Level	Status
TC-1	Pose logging (Android)	No Java unit test	Device	Not yet automated

Table 7.2: Test cases and their status

ID	Description	Evidence	Level	Status
TC-2	IQA blur detection (Android)	No Java unit test; no recorded run	Device	Not yet automated
TC-3	Coverage voxel counting (Android)	No Java unit test; no recorded run	Device	Not yet automated
TC-4	IMU fallback bridging	`test_fallback`, `test_common_fallback`, `test_imu`	Unit	Pass (CI)
TC-5	COLMAP SfM convergence	574-frame v3 model; 1166 gyro-selected frames registered, 1108 kept, BA 0.97 → 0.88 px	System	Pass (manual)
TC-6	Sim3 scale	`test_relocalize_and_sfm`; tape measure (Table 7.5)	Unit + Accept.	Pass (CI, manual)
TC-7	RunPod client	`test_runpod_client` (faked network); no live-pod test	Unit / Integ.	Pass (CI); live run not yet executed
TC-8	Upload endpoint	`test_server_api`	Integ.	Pass (CI)
TC-9	Status polling	`test_server_api`, `test_server_pipeline`	Integ.	Pass (CI)
TC-10	Config switching	`test_server_api`	Integ.	Pass (CI)
TC-11	End-to-end, first room	Local pipeline + tape measure; app→server→RunPod path not recorded	System	Partial
TC-12	End-to-end, second room	SSIM 0.91, 1% badly placed; no tape measurement	System	Partial
TC-13	Reconstruction quality	v6: 26.52 dB / 0.8886 on 42 held-out frames	Accept.	Pass (manual)
TC-14	Processing timeout	Kill path never exercised	Integ.	Not yet executed
TC-15	Pipeline consistency	`test_server_pipeline`	Unit	Pass (CI)
TC-16	Download endpoint	`test_server_api`	Integ.	Pass (CI)

Table 7.2: Test cases and their status

ID	Description	Evidence	Level	Status
TC-17	APK and manifest checks	`test_apk_checks`	Unit	Pass (CI)
TC-18	Coordinate conventions	`test_common_project`, `test_common_capture`	Unit	Pass (CI)
TC-19	Dataset export	`test_process_dataset`	Unit	Pass (CI)
TC-20	Source compilation	`test_sources_compile`	Static	Pass (CI)

7.4 Test Results

Automated tests (CI): The pytest suite has 128 test functions in 12 files (one of them parametrised over every Python file, giving 195 collected tests). Re-run in full on the current `master` head on 22 October 2026: 194 passed, 1 skipped (the APK signing check, which needs `keytool`), ruff clean, in under three seconds. That re-run found three real defects introduced after CI run #6 — including a missing `time` import that made the job-cancel endpoint raise `NameError` — all fixed in commit `0d78466`. In CI run #6 the unit-test step succeeded on Python 3.10, 3.11 and 3.12 and the blocking ruff lint passed. Of the 20 test cases above, 12 are backed by these automated tests (TC-6 is also checked by tape measure), 2 by recorded manual runs (TC-5, TC-13), 2 are only partly executed (TC-11, TC-12), 3 Android cases have no automated test (TC-1 to TC-3), and TC-14 has not been executed. CI runs on every push to non-master branches, on pull requests, and from the CD workflow on `master`; the CI workflow itself has not yet run on the current `master` head, so the multi-version evidence remains that of run #6.

System-level results: The current best model, v6, clears both acceptance thresholds. Gyro-aware frame selection (1166 candidate frames, 1108 kept) contributed the largest single gain (+0.26 dB / +0.009 SSIM); dense MVS initialisation (1.15M fused points reduced to ∼460k at 1 cm) added +0.21 dB / +0.003 SSIM.

Effect of the pose fix. Table 7.4 compares the earlier ARCore-pose pipeline with the image-based pipeline on the same recording.

Table 7.3: Reconstruction quality on the first room: 42 held-out frames that no model was trained on (`fair_compare.py`)

Model	Frames	Initialisation	PSNR (dB)	SSIM
v3 (image-based poses)	574	SfM points	26.04	0.8756
v4_sharp (+ gyro-selected frames)	1108	SfM points	26.30	0.8849
v5 (+ dense MVS initialisation)	1108	dense MVS	26.51	0.8879
v6 (+ finer densification)	1108	dense MVS	**26.52**	**0.8886**
Acceptance target			$\geq$25	$\geq$0.85

Table 7.4: Before and after replacing ARCore poses with image-based (COLMAP) poses

Measure	Before (ARCore poses)	After (v3)
Training-view fit	21.9 dB, 12–30 px misaligned	$\sim$28 dB, 2–3 px
Held-out SSIM	0.78	0.86 (second room: 0.91)
Badly placed surfaces	7–23%	3% (second room: 1%)

Second room. The image-based pipeline was also run on a second room (`roomAttemptSadness2`), measured by Muhammad Tayyab: held-out SSIM 0.91 with 1% badly placed surfaces. A held-out PSNR for the second room has not yet been measured.

Splatfacto versus our own trainer (different scene). On `roomAttemptSadness2`, Nerfstudio splatfacto trained on our preprocessed inputs reached 27.81 dB on held-out views and 28.08 dB on training views, against 24.70 dB held-out for our own trainer with the corrected recipe. This comparison motivated the switch to splatfacto. It is a different scene and protocol from Table 7.3 and is not v6's score.

7.5 Acceptance Testing / Validation

Acceptance testing validates the system against the success criteria defined in Section 1.9:

1. **Metric accuracy**: Four rigid-object distances measured in the model agree with a tape measure to a mean of 4.8 mm (0.19%) and a worst case of 0.29% (Table 7.5), against a 5% target.

2. **Reconstruction quality**: v6 scores 26.52 dB PSNR / 0.8886 SSIM on 42 held-out frames of the first room, clearing the 25 dB / 0.85 thresholds; the second room reaches held-out SSIM 0.91 (its PSNR is not yet measured). Views from the room centre, where no camera stood, are coherent, and the earlier ghosts and floating splats are gone.

3. **Processing time**: The measured end-to-end time is approximately 75 minutes on the development RTX 3070 (feature matching 23 min, mapping 6 min, training 45 min). The NFR-P1 target of under 35 minutes on an RTX 4090 is a projection for FYP-2 and has not yet been measured, so NFR-P1 is not yet verified.

4. **Capture usability**: Not yet formally evaluated. Usability testing with external users is planned for FYP-2.

Tape-measure evidence. Table 7.5 compares distances measured in the reconstructed model (first room, image-based pipeline) with a tape measure. Two further measurements on soft edges (a chair and a rug) are recorded in `docs/progress/Problems_and_Solutions.md`; they are excluded from the rigid-object mean because their endpoints are not well defined.

Table 7.5: Metric accuracy: model versus tape measure (rigid objects)

Object	Model (m)	Tape (m)	Error	Relative
Table (short side)	0.458	0.459	−1 mm	0.22%
Table (long side)	1.748	1.750	−2 mm	0.11%
Wall	2.792	2.800	−8 mm	0.29%
Room length	6.692	6.700	−8 mm	0.12%
Mean			**4.8 mm**	**0.19%**

The worst case is 0.29%, far inside the 5% accuracy target. The error does not grow with distance (−8 mm at both 2.8 m and 6.7 m), so the ARCore-derived scale is sound; the small, consistent negative offset most likely comes from where the endpoints are clicked in the viewer. These are four measurements in one room; more rooms are planned for FYP-2.

7.6 Edge and Failure Cases

1. **Tracking loss during capture**: ARCore loses tracking in featureless corridors or when the phone is moved too fast. The app displays a red "Tracking Lost" banner and pauses quality logging. The IMU fallback (`fallback.py`) bridges the gap; on simulated gaps of 9 s its position error is 3.8 cm against 30.6 cm for interpolation, and re-alignment after re-tracking cuts post-gap error from 49.7 px to 8.2 px

(Table 6.1). In the image-based pipeline COLMAP also registers most gap frames (748 of 759 on the second room). Re-alignment fails if the user never returns to a mapped area; those segments are flagged unverified and kept out of training. *Mitigation*: Live IQA warns the user to slow down; coverage overlay shows unscanned areas.

2. **Plain white walls**: Rooms with large untextured surfaces produce sparse SfM point clouds in those regions. The model shows blurring or holes on plain walls. *Mitigation*: The IQA "low texture" warning prompts the user to capture textured objects nearby.

3. **Mirrors and glass**: Highly reflective surfaces cause incorrect depth estimates and SfM outliers. *Status*: Known limitation; documented for FYP-2 investigation.

4. **Network interruption during upload**: If the HTTP connection drops mid-upload, the app (`ProcessingActivity`) displays an error with a retry button. On the server, an upload missing a required file or carrying a corrupt depth zip is rejected cleanly (TC-8). *Status*: Partly handled; an interrupted upload has not been tested end to end.

5. **RunPod pod failure**: If SSH never becomes ready (pod crash, GPU unavailable), `wait_for_ready` gives up after 300 s (polling every 10 s), the pod is terminated in the `finally` block, and the job is marked failed. *Status*: Implemented; the timeout is unit-tested with a faked API (TC-7), but the termination path has not yet been exercised on a live pod.

7.7 Defects and Corrective Actions

ID	Defect	Root Cause	Corrective Action	Status
D-1	Degenerate splats (zero thickness, half-transparent)	Flattening regularizer × Adam interaction; depth loss gradient conflicts	Switched to splatfacto; removed custom regularizers	Fixed
D-2	5 dB PSNR drop vs. reference implementations	ARCore poses used directly for training (2.3 px reprojection error)	Switched to COLMAP SfM poses (0.62 px)	Fixed

D-3	Ghosting artifacts near doorways	ARCore silent re-anchoring events creating pose discontinuities	Training poses from image-based COLMAP SfM; ARCore anchor logging added so re-anchors are detectable	Fixed
D-4	SfM failure on 30 FPS input	Too many similar frames overwhelm COLMAP exhaustive matching	Keyframe selection: one sharp frame per window (`prepare.py`); gyro-aware selection (`select_sharp.py`) for v4–v6	Fixed
D-5	Barrel distortion visible at frame edges	ARCore intrinsics lack lens distortion coefficients	COLMAP self-calibration recovers k_1, k_2; undistort before training	Fixed

7.8 Current Validation Status

Status of the Iteration 1 acceptance criteria (core pipeline complete, refinement ongoing):

- ✓ Reconstruction quality: v6 scores 26.52 dB / 0.8886 on the first room ($\geq$25 dB, $\geq$0.85). Second room: SSIM 0.91; PSNR not yet measured.
- ✓ Metric accuracy: mean 0.19%, worst case 0.29% against the 5% target (Table 7.5).
- Processing time: *not yet verified*. Measured $\sim$75 min on RTX 3070; the $<$35 min RTX 4090 target (NFR-P1) is projected and not yet measured.
- Pod lifecycle: *partly verified*. `terminate_pod()` is called from `finally` blocks and the client calls are unit-tested with a faked API; a live-pod test is not yet executed.
- ✓ CI pipeline: suite re-run green on the current `master` head (194 passed, 1 skipped, ruff clean); CI run #6 covers Python 3.10–3.12.
- Not yet automated or executed: Android unit tests (TC-1 to TC-3), processing timeout (TC-14), app-to-RunPod end-to-end run.

Remaining validation for FYP-2:

- Measurement accuracy on $\geq$3 additional rooms with varied geometry.
- Structural-layer accuracy: wall lengths against tape measure and detected doors against a manual count.
- Web viewer usability testing with external users.
- Stress testing: large rooms, long recordings ($>$10 minutes), low-end Android devices.

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

1. Android capture app with ARCore pose/depth/quality logging, live IQA, coverage overlay, upload UI.
2. Reconstruction pipeline: IMU fallback, gyro-aware frame selection, COLMAP SfM with Sim3 metric alignment, Nerfstudio splatfacto training, model export.
3. FastAPI processing server with local and RunPod cloud GPU modes.
4. RunPod integration with guaranteed pod lifecycle management.
5. CI/CD pipeline (GitHub Actions): lint, unit tests, APK build, web build, hygiene checks.
6. Extensive root-cause analysis of degenerate splat geometry (6 causes, documented in `Training_Analysis.md`).
7. Server launcher script (`start_server.bat`) with `.env` support.

8.2 Work Remaining

1. **Web viewer**: Three.js/WebGPU Gaussian splat viewer with orbit controls and measurement tools.
2. **Measurement tool**: Point-to-point distance measurement in the metric 3D model.

3. **Structural layer**: Complete `structure.py`: wall-to-wall dimensions, doors and openings, ceiling-height detection, and automatic floor-plan export.
4. **Wall snapping**: Measurement clicks that snap to structural-layer walls, and a floor-plan view in the web viewer.
5. **3DGS refinement**: One-command v6 pipeline, validation on the second room, and IMU-driven motion-blur modelling.
6. **3D mesh creation**: In scope as planned work; not yet scheduled.
7. **Field validation**: Metric accuracy validation on $\geq$3 additional rooms with varied geometry.
8. **User testing**: Usability evaluation with external users (non-team members).

8.3 Individual Responsibilities

The primary evidence of each member's contribution is the set of modules they own: the files whose commit history carries their name (`git log - <file>`). Table 8.1 lists them.

8.4 Contribution of Each Team Member

Muhammad Umair (23I-0662): Project lead. Built the processing server (FastAPI endpoints, job model with per-step timeouts, local pipeline runner) and the RunPod integration (pod creation, SSH readiness, remote pipeline, termination in `finally` blocks). Built the app's upload and progress screen and the scan gallery/detail screens, worked on the main AR activity and coverage overlay, built the web UI prototype, and wrote the proposal and report.

Zaid Fareed (23I-0829): Built the CI/CD infrastructure (lint, test and build workflows on three Python versions, Android build and signing checks, repository hygiene, releases) and wrote the entire automated test suite that backs the "Pass (CI)" results in Chapter 7. Also contributed trainer fixes and the ARKitScenes trainer experiments.

Muhammad Tayyab (23I-0636): Built the reconstruction pipeline: IMU fallback and re-alignment, keyframe and gyro-aware frame selection, bundle adjustment, the COLMAP SfM stage with Sim3 metric alignment, dense MVS initialisation, the structural layer, and the evaluation tools. Wrote the app's capture logging and live IQA, ran the training analysis, and produced the v6 model, the tape-measure validation and the second-room result.

Commit counts. The repository has **133** commits: Muhammad Umair 85, Zaid Fareed 26, Muhammad Tayyab 20 (committing as "BlazingBeam"), and 2 automated commits by the GitHub Actions bot. **These counts reflect differing commit granularity, not differing effort.** Umair commits each small change separately; Tayyab works in large batches, so a single commit of his often carries a whole pipeline stage (for example, the commit that introduced image-based SfM poses and lens self-calibration, or the one that introduced gyro-aware frames, dense MVS initialisation and the structural layer). The owned modules above, not the commit counts, are the evidence of contribution.

8.5 Repository and Version Control Evidence

The project repository is hosted on GitHub. Key statistics:

- **Repository**: `github.com/Umairishuman/FYP` (private)
- **Branches**: `master` (stable) and work branches such as `ci/github-actions`, `ci/pipeline-update`
  and `exp/trainer-arkitscenes`
- **Commit history**: 133 commits (85 / 26 / 20, plus 2 bot commits)
- **CI**: GitHub Actions runs lint, tests and builds on work branches and pull requests
- **Code review**: Pull request template with area checkboxes and a testing checklist

8.6 Milestones and Commit Evidence

8.7 Tools and Tool-Assisted Development

IMPORTANT: Generative AI Tool Disclosure

In compliance with university policy, we disclose the use of the following AI-assisted development tools:

Statement of originality: All core algorithms (Sim3 alignment, gyro-aware frame selection, coverage map, IQA pipeline, RunPod lifecycle management) were designed by the team. AI tools were used as assistants for code generation, debugging, explanation and report drafting, as disclosed in Table 8.3. The system architecture, experimental methodology, root-cause analysis, and design decisions are entirely our own work. All AI-generated code was reviewed, understood, tested, and adapted by team members before integration into the codebase.

Other development tools:

- Android Studio (Ladybug) with Android Gradle Plugin
- Visual Studio Code with Python and Java extensions
- Git + GitHub for version control
- Postman for API testing during server development
- ADB (Android Debug Bridge) for device-level debugging
- NVIDIA Nsight for GPU profiling during training experiments

8.8 External Components and Dependencies

All external components are used under permissive open-source licenses compatible with academic use. No proprietary or restrictively-licensed code is included.

8.9 Risks and Mitigation

Table 8.5 is the project's risk register. Likelihood and impact are rated Low, Medium or High by the team; the owner is the member whose modules the risk mainly affects (Table 8.1). The register is reviewed at the start of each FYP-2 iteration.

Table 8.5: Risk register

ID	Risk	Likel.	Impact	Mitigation	Owner
R1	Cloud GPU cost exceeds budget	Medium	High	Per-step timeouts; `terminate_pod()` in `finally` blocks whenever a pod exists; watchdog thread with a hard time limit; GPU type selectable per job. The cost per scan has not yet been measured; measure it on the first RunPod runs in Iteration 2a.	Umair
R2	Cloud GPU unavailable (RTX 4090 out of stock on RunPod)	Medium	Medium	GPU type selectable at runtime from the list RunPod reports as in stock; existing-pod mode; local RTX 3070 fallback (measured $\sim$75 min per room).	Umair

Table 8.5: Risk register

ID	Risk	Likel.	Impact	Mitigation	Owner
R3	Tracking loss on featureless surfaces (plain walls, ceilings)	High	Medium	Live IQA "low texture" warning and coverage overlay during capture; IMU fallback bridges gaps (3.8 cm vs 30.6 cm over 9 s) and re-aligns after re-tracking (49.7 $\rightarrow$ 8.2 px); image-based SfM registers most gap frames. Segments that cannot be re-aligned are flagged and kept out of training.	Tayyab
R4	Mirrors and glass produce SfM outliers and artefacts	Medium	Medium	Documented limitation; detect and mask reflective regions in FYP-2 (Iteration 2b).	Tayyab
R5	Large rooms exceed GPU memory	Medium	Medium	The 8 GB RTX 3070 already needs default densification thresholds and no page-locked caching above 4 GB. Train large rooms on a 24 GB RTX 4090 (not yet measured); fewer keyframes on slow walks.	Tayyab
R6	RTX 4090 performance is unmeasured; the $<$35 min target may not be met	High	Medium	The target is a projection. Benchmark on an RTX 4090 at the start of Iteration 2a; attack feature matching first (23 of the 75 min measured on the RTX 3070) with ARCore-pose-guided or sequential pairs; report the measured time whatever it is. Note that the RunPod pod watchdog's default hard limit (`MAX_POD_LIFETIME`) is 60 min, shorter than the only measured run; raise it, or confirm the run fits, before the first full RunPod job.	Umair
R7	FYP-2 scope not delivered in time (3DGS refinement, structural layer, web workspace, 3D mesh creation)	Medium	High	Fixed order of work: one-command v6 pipeline $\rightarrow$ web viewer with measurement $\rightarrow$ structural layer and floor plans $\rightarrow$ dashboard. The dashboard is cut first if time runs short. 3D mesh creation stays unscheduled until the core items are done.	Team

Table 8.5: Risk register

ID	Risk	Likel.	Impact	Mitigation	Owner
R8	Measurement evidence too thin (four rigid measurements, one room)	Medium	Medium	Tape-measure comparisons on $\geq$5 rooms in FYP-2, including the second room; structural-layer wall lengths checked against tape.	Tayyab
R9	Only one phone model tested (realme)	Medium	Medium	Record on at least one more ARCore phone in FYP-2; the app logs per-frame exposure and the factory lens calibration, so device differences can be measured.	Umair
R10	Nerfstudio / gsplat breaking changes or a lost local patch	Low	Medium	Nerfstudio pinned to 1.1.5; the local pin-memory patch is documented in `docs/progress/`; GPU Docker image built by CI.	Tayyab
R11	ARCore API changes or deprecation	Low	High	ARCore is used only in the capture app; the pipeline consumes plain files (`dataset.mp4`, `poses.csv`, ...), so another tracker can feed it.	Umair
R12	Team member unavailability	Low	Medium	All code in one shared repository; owned modules documented; progress logs in `docs/progress/`; automated tests guard the Python code.	Team

8.10 Security and Privacy Considerations

- **API key management**: The RunPod API key is stored in a `.env` file excluded from version control via `.gitignore`. The key is loaded at runtime from environment variables, never hardcoded.
- **CI secrets scanning**: The GitHub Actions CI pipeline includes a secrets detection check that blocks merges if API keys, tokens, or credentials are found in committed files.
- **Data privacy**: Captured scan data (video, depth, poses) is transmitted to the processing server over the local network. In production, HTTPS would be required. Scan data is stored only for the duration of processing and can be deleted via the API.

- **Cloud GPU security**: RunPod pods are created on-demand, used for a single job, and terminated immediately after. No persistent data remains on the cloud GPU.

8.11 Current Limitations

1. **Mirrors and glass**: Highly reflective and transparent surfaces cause SfM outliers and incorrect depth. The model shows artifacts in these regions. Planned FYP-2 investigation: detect and mask reflective regions.

2. **Large rooms**: Rooms exceeding approximately 8×8 m may exceed GPU memory during training on the RTX 3070 (8 GB). A 24 GB cloud GPU (RTX 4090, the RunPod default) is expected to handle larger scenes; this has not yet been measured.

3. **Outdoor scenes**: The system is designed for indoor environments. Outdoor scenes with sky, vegetation, and long-range views are out of scope.

4. **Real-time processing**: Processing takes approximately 75 minutes on the development RTX 3070; no cloud-GPU timing has been measured yet. Real-time reconstruction is not feasible with current 3DGS training times.

5. **Single-floor only**: The current coverage map and measurement tools assume a single-floor environment. Multi-floor buildings require manual segmentation.

6. **No web viewer yet**: The 3D model can currently only be viewed in a local desktop viewer. The web-based Three.js viewer is planned for FYP-2.

Table 8.1: Owned modules per team member (from the commit history of each file)

Team Member	**Owned modules**
Muhammad Umair (23I-0662)	Processing server: `server/main.py`, `pipeline.py`, `runpod_client.py`, `remote_gpu.py`. Android upload and app screens: `ProcessingActivity`, `ScanDetailActivity`, `UpdateChecker`; co-author of `HelloArActivity`, `CoverageMap`, `CoverageOverlayView`, `QualityMonitor`. Web UI prototype (`prototype/`). GPU Docker workflow; co-author of the CD workflow. Server and job scripts (`scripts/`). Proposal, report and project documentation.
Zaid Fareed (23I-0829)	CI/CD: `ci.yml`, `cd.yml`, APK/manifest checker (`.github/android/`), `ruff.toml`, `pytest.ini`, pull request template. The whole pytest suite (`tests/`, all 12 files and `conftest.py`). Trainer experiments and fixes (`train.py` checkpoints and depth loss, `mvs.py`), ARKitScenes trainer experiments (`docs/Experiments_ARKitScenes.md`). CI/CD documentation.
Muhammad Tayyab (23I-0636)	Reconstruction pipeline (`src/arcore3dgs/`): `fallback.py`, `imu.py`, `relocalize_pnp.py`, `prepare.py`, `select_sharp.py`, `ba.py`, `sfm_colmap.py`, `export_nerfstudio.py`, `mvs_init.py`, `structure.py`, evaluation tools (`fair_compare.py`, `fit_diag.py`, `diagnose.py`). Capture logging and live IQA in the app: `CaptureLogger`, `QualityMonitor`; co-author of `HelloArActivity` and the coverage overlay. Training analysis (`Training_Analysis.md`), tape-measure validation and second-room evaluation.

Table 8.2: Key milestones with commit evidence

Milestone	**Description**	**Evidence**
M1: Capture	ARCore recording with pose/depth/IMU logging	Commits by Tayyab, Umair
M2: Pipeline	IMU fallback, COLMAP SfM, splatfacto training	Commits by Tayyab; trainer fixes by Zaid
M3: Quality	Live IQA, coverage overlay, gyro-aware selection	IQA and selection by Tayyab; coverage overlay by Umair and Tayyab
M4: Server	FastAPI + RunPod integration, upload UI	Commits by Umair
M5: CI/CD	GitHub Actions workflows, test suite	Commits by Zaid; workflow merges by Umair
M6: Validation	v3–v6 results, tape measure, second room	Commits by Tayyab; numbers in `docs/progress/`

Table 8.3: AI and automated tools used during development

Tool	**Usage**	**Scope / Boundaries**
Claude Code (Anthropic)	Agentic coding assistant, used in local and cloud sessions	Code generation and refactoring, debugging and error diagnosis, CI/CD workflow changes (some commits carry a `Co-Authored-By: Claude` trailer), project documentation, drafting of this report, and the correction pass on this report after supervisor review (checking every figure against `docs/progress/` and the team's decisions, rewriting the test and meeting sections, and drawing the TikZ diagrams). All output was reviewed, tested and edited by team members, who remain responsible for it.
GitHub Copilot	Inline code completion	Used for auto-completion in Android Studio and VS Code. All suggestions were reviewed before acceptance.
ChatGPT (OpenAI)	Research queries, algorithm explanations	Used to understand COLMAP parameters, quaternion math, and coordinate system conventions. No code directly copied.

Table 8.4: External components and their licenses

Component	**Version**	**License**	**Usage**
ARCore SDK	1.44+	Apache 2.0	Pose tracking, depth
COLMAP	4.2	BSD-3	Structure from Motion
Nerfstudio	1.1.5	Apache 2.0	3DGS training
FastAPI	0.110+	MIT	REST API server
OkHttp	4.12.0	Apache 2.0	HTTP client
PyTorch	2.1+	BSD-3	GPU computation
OpenCV	4.9+	Apache 2.0	Image processing

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

FYP-2 has three priorities: (1) finishing the refinement of the reconstruction pipeline and making it faster; (2) completing the structural layer (wall planes, corners, wall lengths, door detection and floor plans) that turns the photoreal model into something that can be measured reliably; and (3) building the web workspace with 1:1 metric measurement. The core capture $\rightarrow$ upload $\rightarrow$ reconstruct pipeline was built in FYP-1; FYP-2 completes and validates it.

9.2 Remaining Modules and Features

1. **Pipeline Optimization Module**: Make the v6 recipe (gyro-selected frames, dense MVS initialisation, finer densification) the one-command pipeline and validate it on the second room. Reduce end-to-end processing time from the measured $\sim$75 min on the development RTX 3070 (matching 23 min, mapping 6 min, training 45 min) to under 35 min on an RTX 4090. The 35 min figure is a projected target, not a measurement. Techniques: ARCore-pose-guided or sequential matching pairs instead of exhaustive COLMAP matching, fewer training iterations, and fewer keyframes on slow walks.

2. **Reconstruction Refinement**: IMU-driven motion-blur and rolling-shutter modelling during training, using the per-frame exposure time and rolling-shutter readout the app now logs (the gyroscope shows the median training frame is blurred by 6–12 px). Also ARCore-started image-only alignment with re-anchor detection from the logged anchors, which could give COLMAP-quality poses faster.

3. **Robustness Improvements**: Better handling of mirrors and glass (detecting and

masking reflective regions), textureless walls, large rooms (>8 m), and variable lighting conditions.

4. **Structural Layer Module**: Complete `structure.py`, which already rebuilds the room shell next to the splat model: floor and wall planes, corners from neighbouring wall intersections, wall lengths (prototype: 6.65–6.68 × 4.45–4.48 m on the first room) and openings, with 5 doors detected automatically. Remaining work: wall-to-wall dimensions, doors and other openings as first-class objects, fixing the ceiling-height detection, automatic floor-plan export, and measurement clicks that snap to walls. The photoreal splat model itself is not modified, because post-hoc flattening destroyed the glossy floor's reflections and crushed door frames.

5. **Web Workspace Module**: Three.js / WebGPU-based Gaussian splat viewer with orbit/pan/zoom controls, running in any modern browser. Loads the `splat.ply` model and the structural layer's `structure.json`, and shows a floor-plan view.

6. **Measurement Module**: Point-to-point 1:1 distance tool in the metric 3D model. The user clicks two points on the splat surface or on a structural-layer wall; the system computes the Euclidean distance in metres in the Sim3-aligned coordinate frame. Target accuracy: within 5% of tape measure (current evidence: mean 0.19%, worst case 0.29%).

7. **3D Mesh Creation**: Create a 3D mesh of the scanned room from the reconstruction, alongside the splat model. This is in scope as planned work; it has not yet been assigned to an iteration.

8. **Dashboard Module**: Web-based project dashboard showing scan history, processing status, model thumbnails, and measurement records.

9.3 Remaining Iterations

- **Iteration 2a (Weeks 1–4)**: Pipeline optimization and web viewer. Deliverable: one-command v6 pipeline; processing time <35 min on RTX 4090 (projected, not yet measured); browser-based splat viewer with point-to-point measurements validated on ≥3 rooms.

- **Iteration 2b (Weeks 5–8)**: Robustness, reconstruction refinement and the structural layer. Deliverable: improved edge-case handling; structural layer with wall lengths, doors and floor-plan export; measurement snapping to walls in the viewer.

- **Iteration 2c (Weeks 9–12)**: Dashboard, field validation, and final polish. Deliverable: complete web workspace, user testing on 5+ rooms, final report.

9.4 Implementation Plan

Iteration 2a: Optimization + Web Viewer (Weeks 1–4)

1. Put the v6 recipe into the one-command pipeline; replace exhaustive matching with ARCore-pose-guided or sequential pairs (week 1).
2. Reduce splatfacto iterations and keyframes on slow walks; benchmark every change on the two reference rooms (week 1–2).
3. Set up a Three.js project with a Gaussian splat renderer and camera controls (week 2).
4. Build the measurement tool: ray-cast click $\rightarrow$ nearest splat $\rightarrow$ 3D coordinate $\rightarrow$ distance display (week 3).
5. Validate measurement accuracy against tape-measure ground truth on 3 rooms (week 3–4).
6. Integrate the viewer with the FastAPI backend for model loading (week 4).

Iteration 2b: Robustness + Structural Layer (Weeks 5–8)

1. Implement reflective-surface detection and masking for mirrors and glass (week 5).
2. Try IMU-driven motion-blur and rolling-shutter modelling on recordings that carry per-frame exposure metadata (week 5–6).
3. Structural layer: fix ceiling-height detection; make doors and openings first-class objects with their dimensions (week 6).
4. Structural layer: wall-to-wall dimensions and automatic floor-plan export from the floor polygon (week 7).
5. Load `structure.json` in the web viewer; snap measurement clicks to walls; add a floor-plan view (week 8).

Iteration 2c: Dashboard + Final Validation (Weeks 9–12)

1. Build the web dashboard: scan list, processing status, model thumbnails (week 9–10).
2. User testing with 5+ external users; collect usability feedback (week 10–11).

3. Fix usability issues, write documentation (week 11–12).

4. Final validation: measurement accuracy, structural-layer accuracy, processing time on 5+ rooms (week 12).

9.5 Testing and Validation Plan

- **Measurement accuracy**: Compare point-to-point measurements in the web viewer against tape-measure ground truth on $\geq$5 rooms of varying size and geometry. Target: error within 5% of tape measure, extending the current four-measurement evidence (mean 0.19%) to more rooms.

- **Structural layer**: For each test room, compare extracted wall lengths against tape measure (target: within 5%) and the detected doors against a manual count.

- **Web viewer performance**: Maintain $\geq$30 FPS on a mid-range laptop (integrated GPU) for models up to 2 million splats.

- **Usability testing**: Task completion rate $\geq$90% for first-time users performing capture, viewing, and measurement tasks.

- **Processing time**: End-to-end pipeline $<$35 min on RTX 4090 (projected target; currently $\sim$75 min measured on RTX 3070). Benchmark each optimization individually.

- **Cross-browser**: Verify viewer on Chrome, Firefox, and Edge.

9.6 Deployment Plan

1. **Server**: Deploy the FastAPI backend and web viewer on a VPS with HTTPS (Let's Encrypt). Static assets served via CDN.

2. **GPU processing**: Continue using RunPod on-demand pods. Evaluate RunPod serverless endpoints if job volume increases.

3. **Android app**: Distribute the APK via GitHub Releases. Google Play Store submission if time permits.

4. **CI/CD**: Extend the existing GitHub Actions to deploy the web frontend on push to master.

9.7 Tentative Timeline

Table 9.1: FYP-2 tentative timeline

Week	Task	Deliverable
1–2	One-command v6 pipeline, matching and training speed-ups + Three.js viewer setup	Processing <35 min (projected); viewer MVP
3–4	Measurement tool + accuracy validation on 3 rooms	Measurement accuracy report
5–6	Robustness (mirrors, textureless walls), motion-blur modelling, structural-layer fixes (ceiling height, openings)	Robust pipeline; structural layer v2
7–8	Floor-plan export, wall snapping and floor-plan view in the viewer	Measurable web workspace
9–10	Dashboard + user testing (5+ external users)	Web platform MVP
11–12	Final validation on 5+ rooms, polish, final report	Complete system

9.8 Expected Final Deliverables

1. **Android APK**: Capture app with live IQA, coverage overlay, and upload.
2. **Web workspace**: Gaussian splat viewer with 1:1 measurement tools and a floor-plan view.
3. **Processing server**: FastAPI backend with local and cloud GPU support.
4. **Structural layer**: Per-room `structure.json` (planes, corners, wall lengths, doors and openings) and floor-plan export.
5. **Validation report**: Measurement accuracy on $\geq$5 rooms, structural-layer accuracy, processing-time benchmarks, usability test results.
6. **Documentation**: User guide, API documentation, deployment guide.
7. **Source code**: Complete repository with CI/CD, tests, and documentation.
8. **Final report**: FYP-2 report with complete results and analysis.

Chapter 10

Conclusion

This report presents the mid-evaluation status of BuildScan 3D, a system that converts casual smartphone video into metrically accurate 3D Gaussian Splatting models for indoor construction inspection.

Problem. Existing 3D scanning solutions either require expensive hardware (Matterport, $6,000+), lack metric scale (Luma AI, PocketGS), or demand expert operation (RealityCapture, manual COLMAP). No current system provides a fully automated phone-to-measurement pipeline with real-time capture guidance.

Research. Our literature review and empirical experiments established four key findings: (1) COLMAP SfM poses outperform ARCore poses for 3DGS training (training-view fit 21.9 dB $\rightarrow$ $\sim$28 dB and held-out SSIM 0.78 $\rightarrow$ 0.86 once ARCore's silent re-anchor and missing lens model were removed); (2) dense MVS initialization is critical for splat geometry; (3) aggressive regularization degrades quality when combined with Adam; (4) gyro-aware frame selection outperforms texture-based sharpness metrics.

Design and implementation. We built a three-tier system—Android capture app, FastAPI processing server, and cloud GPU pipeline—implementing all Iteration 1 features: ARCore capture with live IQA and coverage overlay, COLMAP SfM with Sim3 metric alignment, Nerfstudio splatfacto training, RunPod cloud GPU integration with guaranteed pod lifecycle management, and CI/CD via GitHub Actions.

Validation. The current best model (v6) scores 26.52 dB PSNR ($\geq$25 dB target) and 0.8886 SSIM ($\geq$0.85 target) on 42 held-out frames. Tape-measure checks give a mean error of 4.8 mm (0.19%) on rigid objects and a worst case of 0.29%, substantially better

than the 5% accuracy target. Processing currently takes about 75 minutes on an RTX 3070; the under-35-minute RTX 4090 target is projected and not yet measured.

Risks. The risk register (Table 8.5) tracks twelve risks. The highest are cloud GPU cost (mitigated by timeouts, pod termination and a watchdog; cost per scan not yet measured), the unmeasured RTX 4090 processing time against the projected <35 min target, tracking loss on featureless surfaces (mitigated by live guidance and the IMU fallback), and delivering the remaining FYP-2 scope. Mirrors and glass remain a known limitation.

FYP-2 plan. The remaining work focuses on making the 3D model actionable: a Three.js web viewer with point-to-point 1:1 measurement tools, completion of the structural layer (wall planes, corners, wall lengths, door detection) with automatic floor plans, and field validation on multiple rooms. 3D mesh creation is also in scope as planned work and has not yet been scheduled. At mid-evaluation, Iteration 1 stands at *core pipeline complete, refinement ongoing*. The foundation built in FYP-1—metric accuracy, automated pipeline, and capture quality guidance—provides a solid base for these extensions.

Appendix A

Proposal Action Register

The following table records all feedback received during the FYP proposal defense and the actions taken in response.

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
1	How does this differ from Polycam or Luma AI?	Clearly differentiate from consumer apps	Added metric scale via ARCore Sim3 alignment (tape-measure error: mean 0.19%, worst 0.29%), live IQA capture guidance, and measurement tools. Created comparison table.	Done	Chapter 2, Table 2.1

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
2	Show quantitative accuracy, not just visual quality	Provide metric accuracy measurements	Tape-measure comparison on four rigid measurements: mean error 4.8 mm (0.19%), worst case 0.29%, against a 5% target. PSNR/SSIM reported on 42 held-out frames.	Done	Tables 7.3 and 7.5
3	Define measurable acceptance criteria	Set quality thresholds	Acceptance criteria: PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85, measurement error $\leq$ 5% of tape measure, processing $<$ 35 min on RTX 4090 (projected FYP-2 target; 75 min measured on RTX 3070)	Done	Section 1.9, Table 4.3
4	The pipeline should be automated, not manual	Show fully automated pipeline	Built FastAPI server with RunPod cloud GPU integration; one-tap upload from Android app triggers entire pipeline automatically	Done	Chapter 6, Section 6.5

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
5	Consider cost implications of cloud GPU	Address cloud compute costs	Per-step timeouts prevent unbounded billing; guaranteed pod termination in try/finally; GPU type selectable at runtime via `/config` (default RTX 4090)	Done	Risk register, Table 8.5 (R1, R2)

Appendix B

Detailed Requirements

B.1 Extended Functional Requirements

B.1.1 Mobile Capture Module — Detailed

ID	Description	Priority	Dependency	Status
FR-M1	Record video at 30 FPS with per-frame camera poses (position, quaternion, intrinsics)	Must	ARCore SDK	Done
FR-M2	Log tracking state and failure reasons per frame	Must	FR-M1	Done
FR-M3	Save ARCore feature points with confidence	Should	ARCore SDK	Done
FR-M4	Save depth maps at 5 Hz with confidence	Should	ARCore Depth API	Done
FR-M5	Compute and display live blur, brightness, motion, texture metrics	Must	FR-M1	Done
FR-M6	Display voxel-based coverage overlay (red/yellow/green)	Must	FR-M4	Done
FR-M7	Upload complete dataset via multipart HTTP POST	Must	Server available	Done
FR-M8	Configurable server URL via settings menu	Must	–	Done
FR-M9	Display processing progress (step, %, time, logs)	Must	FR-S5	Done

B.1.2 Processing Server Module — Detailed

ID	Description	Priority	Dependency	Status
FR-S1	Accept multipart uploads (video, poses, tracking, points, quality, meta, depth.zip)	Must	–	Done
FR-S2	Create unique job ID and track status	Must	–	Done
FR-S3	Run pipeline with per-step timeout watchdogs	Must	Pipeline scripts	Done
FR-S4	Support local GPU and RunPod cloud GPU modes	Must	RunPod API key	Done
FR-S5	Expose progress-polling endpoint	Must	FR-S2	Done
FR-S6	Provide download endpoint for splat.ply	Must	FR-S3	Done
FR-S7	Always terminate RunPod pod after processing	Must	FR-S4	Done

B.1.3 Reconstruction Pipeline Module — Detailed

ID	Description	Priority	Dependency	Status
FR-P1	Extract keyframes using gyro-aware sharpness prediction	Must	Exposure metadata	Partial: `select_sharp.py` validated (v4–v6), not yet in the one-command pipeline
FR-P2	Run COLMAP SfM with SIFT, self-calibration, global mapper	Must	COLMAP 4.2	Done
FR-P3	Compute Sim3 alignment to ARCore metric frame	Must	FR-P2, FR-M1	Done
FR-P4	Train splatfacto and export splat.ply	Must	Nerfstudio 1.1.5	Done

FR-P5	Bridge tracking gaps using IMU dead reckoning	Should	FR-M2	Done

B.2 Extended Non-Functional Requirements

ID	Category	Description	Measurement
NFR-R1	Reliability	Pod terminated within 60 s of completion/failure	Server log timestamps
NFR-R2	Reliability	Each step has timeout watchdog	Pipeline config file
NFR-U1	Usability	Real-time visual feedback without 3D knowledge	User testing with external users (planned for FYP-2; not yet executed)
NFR-U2	Usability	Progress display with step, %, elapsed time	UI inspection
NFR-P1	Performance	E2E processing $<$ 35 min on RTX 4090 (projected FYP-2 target)	Elapsed time on an RTX 4090 pod; not yet measured (75 min measured on RTX 3070)
NFR-P2	Performance	30 FPS during recording, no drops	logcat frame timing
NFR-P3	Performance	Recording fast-path skips non-essential rendering	GL thread profiling
NFR-S1	Security	API key from env vars, never hardcoded	Code review + CI secrets check
NFR-S2	Security	.env excluded from version control	.gitignore inspection

B.3 Full Traceability Matrix

Req.	Module	Design Component	Implementation File	Test ID
FR-M1	Mobile	CaptureLogger	CaptureLogger.java	TC-1

FR-M2	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M3	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M4	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M5	Mobile	QualityMonitor	QualityMonitor.java	TC-2
FR-M6	Mobile	CoverageMap	CoverageMap.java	TC-3
FR-M7	Mobile	ProcessingActivity	ProcessingActivity.java	TC-8
FR-M8	Mobile	SettingsMenu	HelloArActivity.java	–
FR-M9	Mobile	ProcessingActivity	ProcessingActivity.java	TC-9
FR-S1	Server	/upload endpoint	main.py	TC-8
FR-S2	Server	Job dataclass	pipeline.py	TC-9
FR-S3	Server	StepConfig timeouts	pipeline.py	TC-15, TC-14
FR-S4	Server	RunPodClient	runpod_client.py	TC-7
FR-S5	Server	/status endpoint	main.py	TC-9
FR-S6	Server	/download endpoint	main.py	TC-16
FR-S7	Server	try/finally termination	runpod_client.py	TC-7
FR-P1	Pipeline	prepare.py / select_sharp.py	prepare.py, select_sharp.py	TC-5
FR-P2	Pipeline	sfm_colmap.py	sfm_colmap.py	TC-5, TC-18
FR-P3	Pipeline	Sim3 alignment	sfm_colmap.py	TC-6
FR-P4	Pipeline	ns-train + ns-export	pipeline.py	TC-13
FR-P5	Pipeline	fallback.py	fallback.py	TC-4

Appendix C

Detailed Test Cases

The automated tests are the pytest suite in `tests/`: 12 test files plus a shared `conftest.py` that builds small synthetic ARCore capture folders (no GPU, no network, no phone). The suite tests the Python side of the project only. **There are no Android (Java) unit tests**; the capture app is checked on a phone and, for its packaging, by the APK/manifest checker tests.

Where "Pass (CI)" comes from. The suite was re-run in full against the current `master` head (commit `0d78466`, 22 October 2026): **194 passed, 1 skipped**, with the blocking ruff lint clean. The single skip is the APK signing check, which needs `keytool` from a local Java installation. Earlier, GitHub Actions CI run #6 (12 October 2026, commit `c3bfcc9` on branch `ci/pipeline-updates`) ran the same suite on Python 3.10, 3.11 and 3.12 and the "Unit tests" step succeeded on all three; the re-run above was on a single interpreter, so the cross-version evidence is still CI's. "Pass (manual)" means a recorded run with its numbers in `docs/progress/`. Anything else is marked "Not yet executed" or "Not yet automated".

Faults found by that re-run. The re-run was not a formality: it found three defects introduced after CI run #6 by the RunPod GPU-selection work. `server/main.py` called `time.time()` in the job-cancel endpoint without importing `time`, so `POST /jobs/{job_id}/cancel` raised `NameError` at runtime; two test files had also gone stale against the widened `/config` payload and the new GPU-name resolution step. All three are fixed in commit `0d78466`. This is itself evidence for the value of the automated suite and the blocking lint gate.

C.1 Automated test files

File	Tests	What it checks
`test_fallback.py`	17	`fallback.py` IMU bridging maths on synthetic data: pose loading, tracking-loss intervals, smoothing, gyro integration and bias, bridged gaps that land on the end anchor with the right velocity, rotation blending.
`test_common_fallback.py`	12	`common.Capture` with fallback output: fused poses used when present, bridged intervals marked, interpolated rotations stay valid rotations.
`test_imu.py`	7	`imu.py` protobuf parsers that read gyroscope/accelerometer samples from the ARCore recording, including truncated or garbage input.
`test_common_capture.py`	11	Reading `poses.csv`, `points.csv` and `tracking.csv`: interpolation between samples, no pose inside a tracking gap, intrinsics, frame-time offsets.
`test_common_project.py`	6	OpenGL→OpenCV conversion and pinhole projection (points behind or outside the image rejected).
`test_relocalize_and_sfm.py`	6	Rigid segment correction (`relocalize.py`) and the Umeyama Sim3 fit used by `sfm_colmap.py`: recovers a known scale/rotation/translation and returns a proper rotation.
`test_process_dataset.py`	3	PLY export and `transforms.json` generation from a synthetic capture.
`test_server_api.py`	19	FastAPI endpoints with the two job starters replaced by recorders: health, config (API key never returned, mode/GPU switching), upload (required files, depth zip, corrupt zip), status, jobs list, download (404/409/200).
`test_server_pipeline.py`	12	`server/pipeline.py`: six steps in order, each with a label and positive timeout, iteration count reaches the training command, `Job.to_dict()` fields, log truncation; the RunPod command list matches the local pipeline step for step.
`test_runpod_client.py`	14	`server/runpod_client.py` with network and subprocesses faked: GraphQL queries and errors, pod creation, SSH-port discovery, `wait_for_ready` success and timeout, `terminate_pod`, ssh/scp command construction.

File	Tests	What it checks
`test_apk_checks.py`	20	The APK/manifest checker used by CI/CD: min SDK, package name, permissions, AR feature, processing screen, signing certificate, APK size; the real manifest passes. This checks packaging, not app logic.
`test_sources_compile.py`	1*	Every Python file in the repository compiles. *Parametrised over every `.py` file.

C.2 Test cases

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-1	Pose logging completeness (`CaptureLogger.java`)	No Android unit test exists. The Python reader of `poses.csv` is covered by `test_common_capture.py`.	One pose row per tracked frame with valid position, quaternion and intrinsics	Device (manual)	Not yet automated
TC-2	IQA blur detection (`QualityMonitor.java`)	No Android unit test exists; no recorded on-device test run.	Blur warning shown on blurred frames	Device (manual)	Not yet automated
TC-3	Coverage voxel counting (`CoverageMap.java`)	No Android unit test exists; no recorded on-device test run.	Voxel direction counts drive red/yellow/green dots	Device (manual)	Not yet automated
TC-4	IMU fallback bridging	`test_fallback.py`, `test_common_fallback.py`, `test_imu.py` (36 tests, synthetic data)	Bridged poses land on the end anchor; rotations stay valid; gyro bias removed	Unit	Pass (CI)
TC-5	COLMAP SfM convergence	First room: v3 trained on 574 frames; 1166 gyro-selected frames registered into the v3 model	Frames register; reprojection error <1.0 px	System	Pass (manual): 1108 of 1166 kept; global BA 0.97 $\rightarrow$ 0.88 px

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-6	Sim3 scale	`test_relocalize_and_known_sim3` (Umeyama fit on synthetic data); scale checked against a tape measure	Known Sim3 recovered; model distances within 5% of tape	Unit + Accept.	Pass (CI); tape: mean 0.19%, worst 0.29%
TC-7	RunPod client	`test_runpod_client.py` with faked network	Correct GraphQL calls, SSH discovery, timeout raised, terminate call sent	Unit	Pass (CI)
		Live run on a real RunPod pod, including termination on failure	Pod terminated in every path	Integ.	Not yet executed
TC-8	Upload endpoint	`test_server_api.py`: multipart upload, missing file, depth zip, corrupt zip	Files saved, job started; bad uploads rejected cleanly	Integ.	Pass (CI)
TC-9	Status polling	`test_server_api.py`, `test_server_pipeline.py`	Status has the fields the app reads; unknown job is 404	Integ.	Pass (CI)
TC-10	Config switching	`test_server_api.py`: POST then GET `/config`	Mode and GPU switch; unknown mode ignored; API key never returned	Integ.	Pass (CI)
TC-11	End-to-end, first room	Capture `newRoomAttemptSadness` → local pipeline → model; distances vs tape	`splat.ply` produced; distances within 5% of tape	System	Pass (manual) for the local pipeline. App upload → server → RunPod path: not yet executed as a recorded test

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-12	End-to-end, second room	Capture `roomAttemptSadness2` → local pipeline → model	Model produced; held-out SSIM ≥ 0.85	System	Pass (manual): SSIM 0.91, 1% badly placed surfaces. Tape measurement: not yet done
TC-13	Reconstruction quality	Evaluate the v6 model on the 42 held-out frames of the first room (`fair_compare.py`)	PSNR $\geq$ 25 dB (actual: 26.52); SSIM $\geq$ 0.85 (actual: 0.8886)	Accept.	Pass (manual)
TC-14	Processing timeout	Set a step timeout below its run time; run the pipeline	Step killed; job status "failed"; error mentions timeout	Integ.	Not yet executed
TC-15	Pipeline consistency	`test_server_pipeline.py`	Spy steps in order with positive timeouts; RunPod and local pipelines match; called scripts exist	Unit	Pass (CI)
TC-16	Download endpoint	`test_server_api.py`	404 unknown job; 409 unfinished or failed job; file returned when finished	Integ.	Pass (CI)
TC-17	APK and manifest checks	`test_apk_checks.py`	Wrong SDK, package, permission, signer or size reported; real manifest passes	Unit	Pass (CI)
TC-18	Coordinate conventions	`test_common_project.py`, `test_common_capture.py`	GL→CV flips y and z; projection and pose interpolation correct	Unit	Pass (CI)
TC-19	Dataset export	`test_process_dataset.py`	PLY written with header; `transforms.json` built	Unit	Pass (CI)

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-20	Source compilation	`test_sources_compile.py`	Every Python file compiles	Static	Pass (CI)

Appendix D

Repository and Contribution Evidence

D.1 Repository Structure

Path	Contents
googleARCore/samples/ hello_ar_java/	Android capture app; Java sources in app/src/main/java/.../helloar/
HelloArActivity.java	main AR activity, recording, settings
CaptureLogger.java	sensor data recording
QualityMonitor.java	live IQA analysis
CoverageMap.java, CoverageOverlayView.java	voxel coverage tracker and dot overlay
ProcessingActivity.java	upload and progress UI
GalleryActivity.java, ScanDetailActivity.java	saved scans
src/arcore3dgs/	reconstruction pipeline
fallback.py, imu.py, relocalize_pnp.py	IMU tracking-gap bridging and post-gap re-alignment
prepare.py, select_sharp.py	keyframe extraction; gyro-aware frame selection
sfm_colmap.py, export_nerfstudio.py	COLMAP SfM + Sim3 alignment; Nerfstudio export
mvs_init.py, structure.py	dense MVS initialisation; structural layer evaluation tools
fair_compare.py, fit_diag.py	
server/	processing server: main.py, pipeline.py, runpod_client.py, remote_gpu.py

Path	Contents
`.github/workflows/`	CI/CD: `ci.yml`, `cd.yml`, `docker-gpu.yml`
`scripts/`	launchers: `start_server.bat`, `view_splat.bat`, job tools
`tests/`	pytest suite (12 files + `conftest.py`)
`prototype/web-ui/`	web UI prototype
`docs/progress/`	`TODO.md`, `Training_Analysis.md`, `Problems_and_Solutions.md`
`.env`	API keys (gitignored)

D.2 Commit History Summary

The repository contains 133 commits (`git shortlog -sne -all`): Muhammad Umair 85, Zaid Fareed 26, Muhammad Tayyab 20 (as "BlazingBeam"), and 2 GitHub Actions bot commits. The counts reflect commit granularity rather than effort: Umair commits each small change, Tayyab commits whole pipeline stages in large batches. Owned modules are listed in Table 8.1. Key commit categories:

- **Muhammad Umair**: processing server, RunPod integration, upload and gallery screens, AR activity and coverage overlay work, web UI prototype, proposal, report and documentation.
- **Zaid Fareed**: CI/CD workflows (`ci.yml`, `cd.yml`), APK checker, the pytest suite, pytest and ruff configuration, pull request template, trainer fixes and ARKitScenes experiments.
- **Muhammad Tayyab**: reconstruction pipeline (`src/arcore3dgs/`), capture logging and live IQA in the app, training analysis, v3–v6 models, tape-measure validation and the structural layer.

D.3 Branch Strategy

- `master`: Stable, CI-passing code. All merges go through pull requests.
- Feature branches (e.g., `ci/github-actions`): Short-lived branches for specific work items, merged via PR with CI checks.

D.4 CI/CD Evidence

The GitHub Actions CI pipeline runs on every push and pull request:

1. **Python checks**: ruff lint + pytest on Python 3.10, 3.11, 3.12.
2. **Web build**: Node.js build of the web UI (placeholder for FYP-2).
3. **Android build**: Gradle APK build.
4. **Hygiene**: File size limits, secrets detection, workflow lint.

The CD pipeline triggers on push to master (build artifacts) and on version tags (GitHub Release with APK and web bundle).

D.5 Task Tracking

Development progress is tracked in `docs/progress/TODO.md`, which serves as the project's task log with dated entries, status markers, and links to relevant commits and analysis documents.

Appendix E

Additional Design Diagrams

The main design diagrams are in the body of the report: the use case diagram (Figure 4.1), the system architecture (Figure 5.1), the job state machine (Figure 5.2), the data flow (Figure 5.3) and the upload–process–download sequence (Figure 5.4). This appendix adds four more detailed diagrams. Each is drawn from the code it names.

E.1 Pipeline Step Sequence

E.2 Android App Architecture

E.3 Sim3 Alignment

E.4 Coverage Map

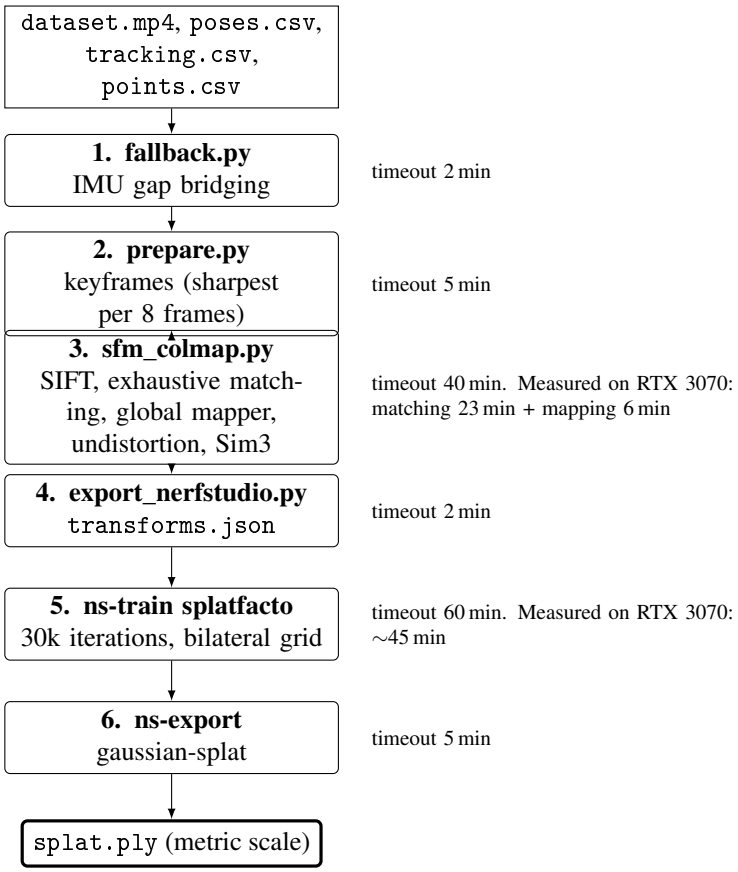


Figure E.1: The six pipeline steps and their timeouts (`STEPS` in `server/pipeline.py`). The measured end-to-end time on the development RTX 3070 is ∼75 min; no cloud-GPU time has been measured yet.

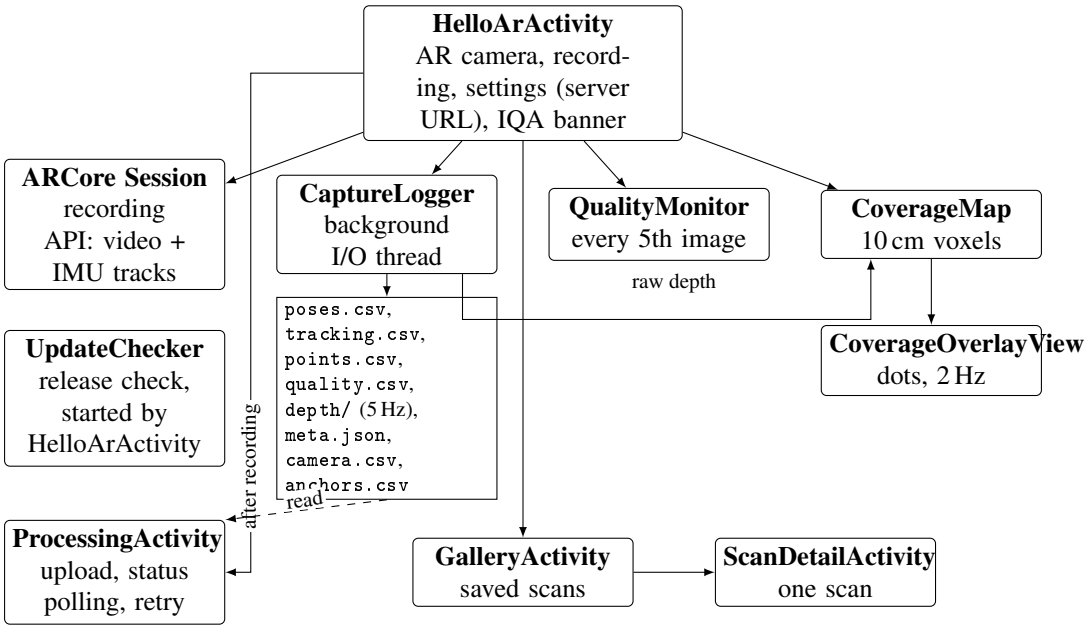


Figure E.2: Android capture app (`googleARCore/samples/hello_ar_java/.../helloar/`). Arrows from `HelloArActivity` show ownership or navigation (`Intents`); `CaptureLogger` feeds raw depth to `CoverageMap`. `ProcessingActivity` uploads six files plus a zipped `depth/` folder; `camera.csv` and `anchors.csv` stay on the phone.

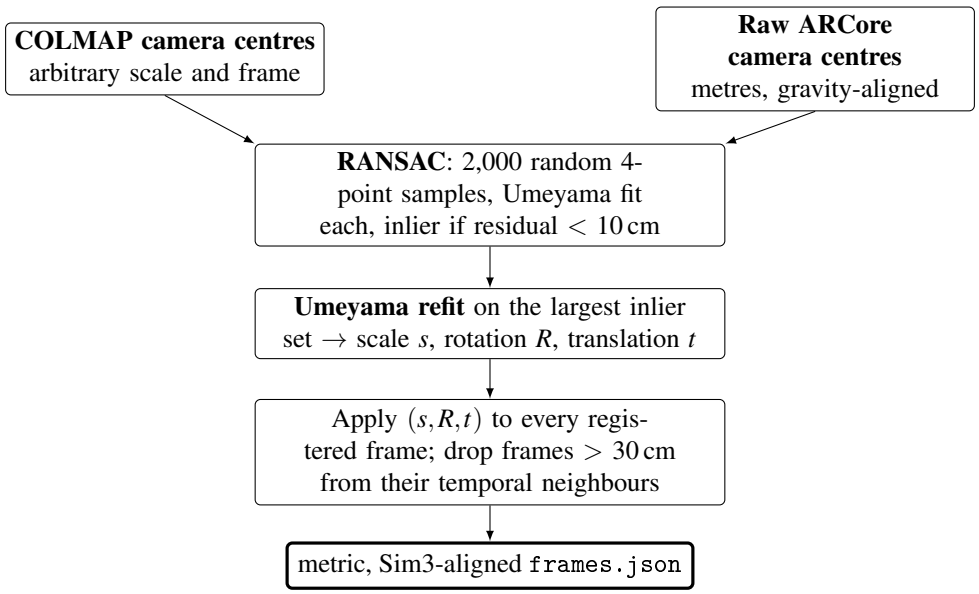


Figure E.3: Sim3 alignment of the COLMAP reconstruction to ARCore's metric frame (`sfm_colmap.py`, export stage; Umeyama fit unit-tested in `test_relocalize_and_sfm.py`). Because ARCore re-anchors silently, only its largest self-consistent segment ends up as inliers. Inlier median residual: about 2.5 cm. The resulting scale is validated against a tape measure (Table 7.5).

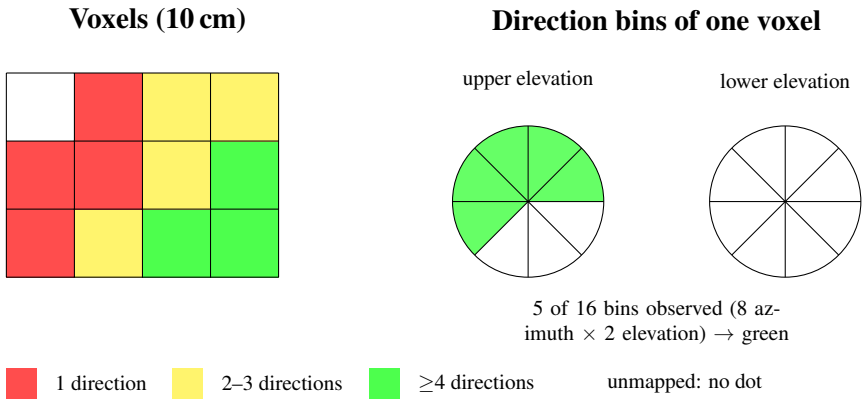


Figure E.4: Live coverage map (`CoverageMap.java`, `CoverageOverlayView.java`). ARCore raw depth (5 Hz, confidence $\geq$128, up to 5 m) is accumulated in 10 cm voxels; each voxel records from which of 16 directions it was seen. Visible voxels are drawn as dots, coloured by direction count, with at most 5,000 voxels projected and 2,000 dots drawn, refreshed at 2 Hz. The grid colouring is illustrative.

Appendix F

Meeting and Progress Evidence

F.1 Supervisor Meetings

The team held **two** meetings with the supervisor, Mr. Furqan Nasir, before the mid-evaluation. Their dates were not recorded at the time, so they are listed in order rather than by date. No formal minutes were taken, and no decision in this report is presented as having been approved by the supervisor.

Meeting	Agenda	Attendees
First meeting	Showing the work completed to date; the techniques used; clarifying the expectations for the mid-evaluation.	Project team and supervisor
Second meeting	Showing the work completed to date; the techniques used; clarifying the expectations for the mid-evaluation.	Project team and supervisor

The dated record of the work itself is the progress log below and the commit history (Chapter 8).

F.2 Progress Log

Detailed development progress is maintained in `docs/progress/TODO.md`. Key milestones:

- **ARCore capture app**: Basic recording → pose sync validation → IQA → coverage overlay → upload UI. Incremental feature additions documented with timestamps.

- **Reconstruction pipeline**: Best custom-trainer model on ARCore-derived poses (`gs_colmap_nc`, 22.67 dB / 0.782 held-out) → image-based COLMAP poses (training-view fit 21.9 → ∼28 dB; held-out SSIM 0.78 → 0.86) → v3 (26.04 dB / 0.8756 on the 42-frame held-out set) → gyro-aware frame selection (+0.26 dB / +0.009 SSIM) → dense MVS initialisation (+0.21 dB / +0.003 SSIM) → v6 (26.52 dB / 0.8886). Each improvement documented with before/after metrics.
- **Quality debugging**: Two-week root-cause analysis documented in `Training_Analysis.md` covering 6 identified causes of degenerate splats, with per-cause experiments and metrics.
- **Server and cloud**: FastAPI server → local pipeline runner → RunPod integration → guaranteed pod lifecycle. Problem-solution pairs documented in `Problems_and_Solutions.md`.

F.3 Problems and Solutions Summary

25 problems encountered during development are documented in `docs/progress/Problems_and_Soluti`
The most significant:

1. ARCore silent re-anchoring causing pose discontinuities (solved: anchor logging + IMU fallback)
2. COLMAP lens distortion not matching ARCore intrinsics (solved: self-calibration)
3. Degenerate splat geometry from custom training recipe (solved: adopted splatfacto)
4. 30 FPS input overwhelming COLMAP matching (solved: keyframe selection, one sharp frame per window)
5. Motion blur in training frames, 6–12 px median (solved for v4–v6: gyro-aware frame selection, +0.26 dB / +0.009 SSIM)
6. GPU memory overflow on the 8 GB RTX 3070 with >1000 full-resolution frames (solved: skip page-locked image caching above 4 GB; default densification thresholds)

Bibliography

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," in *ACM Transactions on Graphics (SIGGRAPH)*, vol. 42, no. 4, 2023.

[2] J. Kim, J. Park, D. Shin, and Y. M. Kim, "Pocketgs: On-device 3d gaussian splatting training and rendering," *arXiv preprint arXiv:2503.10435*, 2025.

[3] Z. Zheng *et al.*, "Vigs-slam: Visual-inertial gaussian splatting slam," in *arXiv preprint arXiv:2411.00483*, 2024.

[4] D. Chen, H. Li, W. Ye, Y. Wang, W. Xie, S. Zhai, N. Wang, H. Liu, H. Bao, and G. Zhang, "Pgsr: Planar-based gaussian splatting for efficient and high-fidelity surface reconstruction," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.